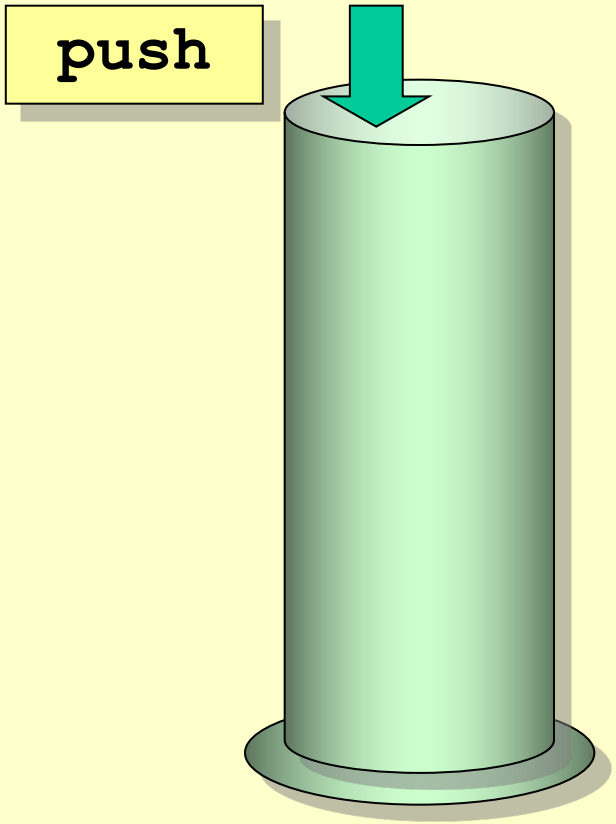
กองซ้อน

(Stack)

การเพิ่ม/ลบข้อมูลในกองซ้อน

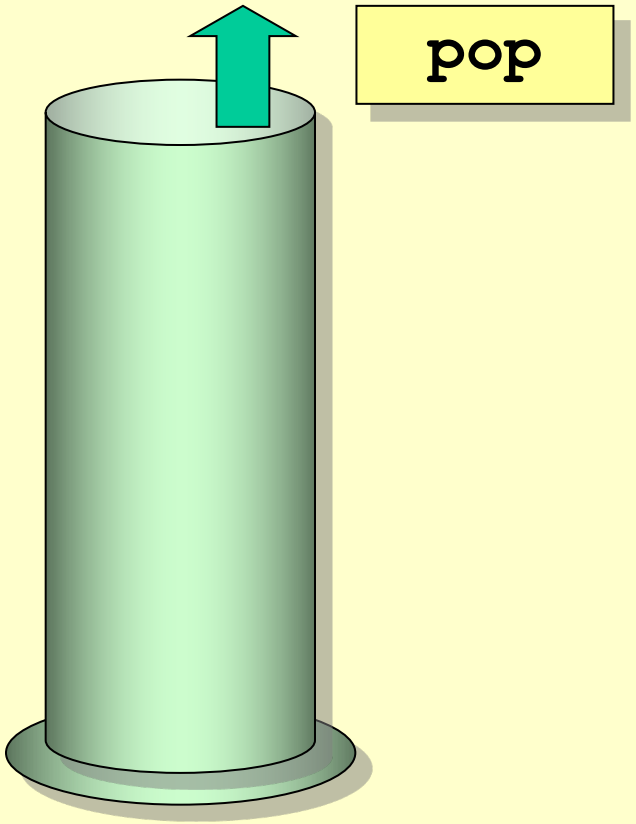
- ข้อมูล เข้าหลัง ออกก่อน (Last-In First-Out)



© S. Prasitjutrakul 2013

การเพิ่ม/ลบข้อมูลในกองซ้อน

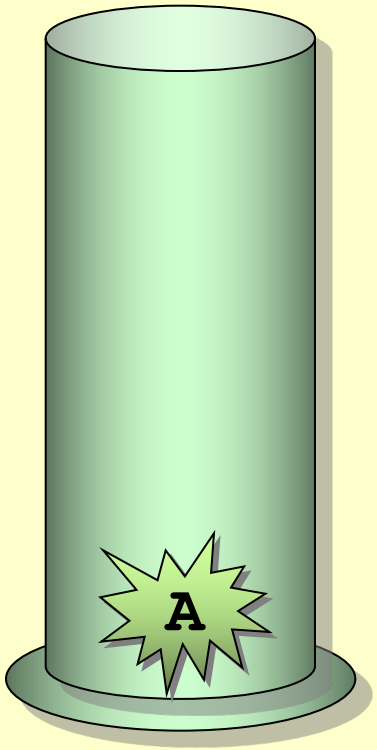
- ข้อมูล เข้าหลัง ออกก่อน (Last-In First-Out)



© S. Prasitjutrakul 2013

การเพิ่ม/ลบข้อมูลในกองซ้อน

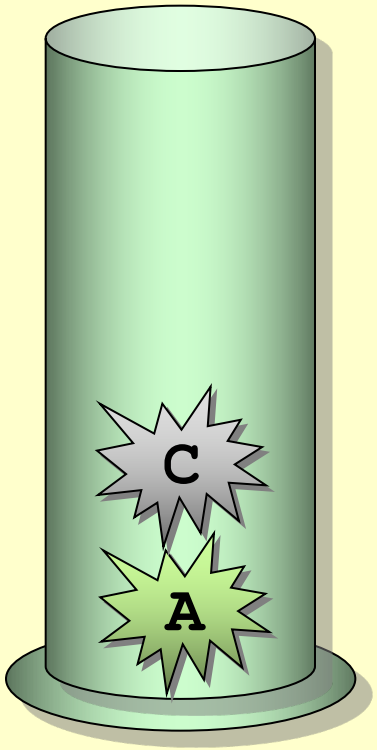
- ข้อมูล เข้าหลัง ออกก่อน (Last-In First-Out)



© S. Prasitjutrakul 2013

การเพิ่ม/ลบข้อมูลในกองซ้อน

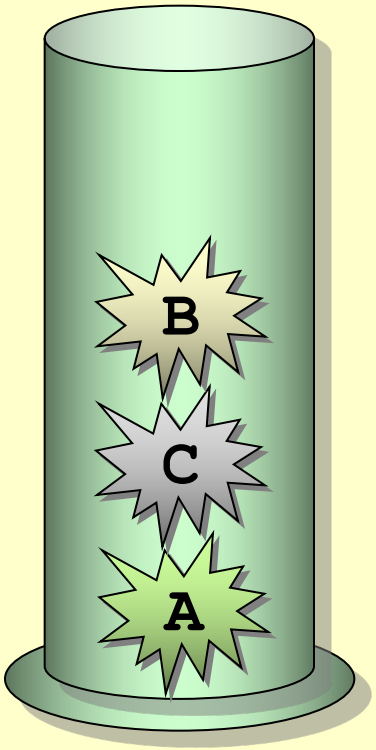
- ข้อมูล เข้าหลัง ออกก่อน (Last-In First-Out)



© S. Prasitjutrakul 2013

การเพิ่ม/ลบข้อมูลในกองซ้อน

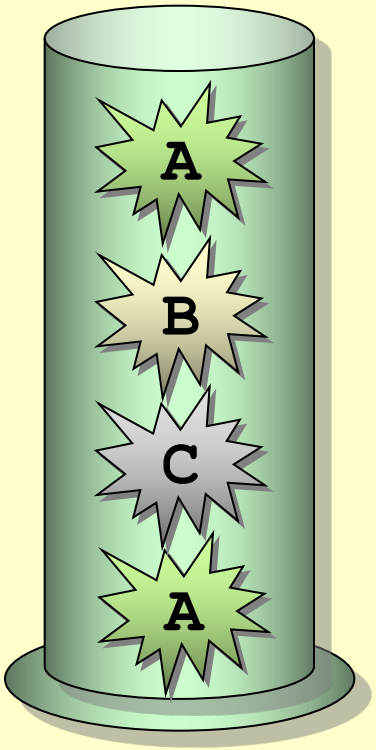
- ข้อมูล เข้าหลัง ออกก่อน (Last-In First-Out)



© S. Prasitjutrakul 2013

การเพิ่ม/ลบข้อมูลในกองซ้อน

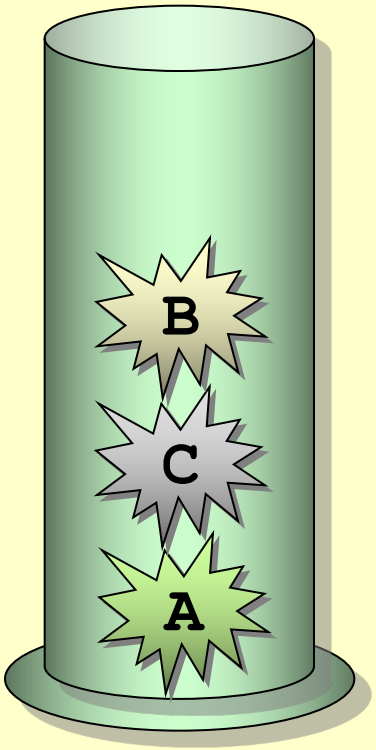
- ข้อมูล เข้าหลัง ออกก่อน (Last-In First-Out)



© S. Prasitjutrakul 2013

การเพิ่ม/ลบข้อมูลในกองซ้อน

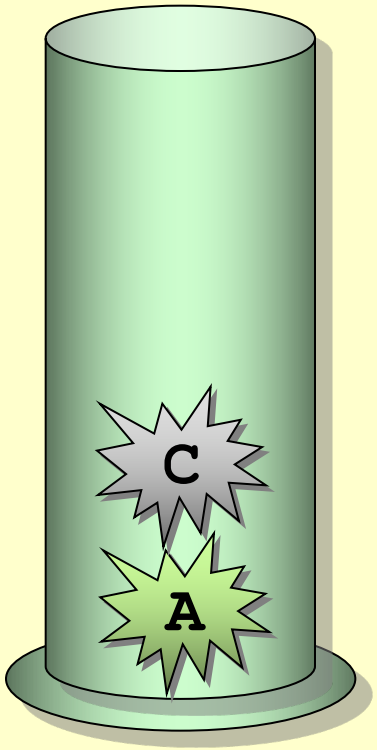
- ข้อมูล เข้าหลัง ออกก่อน (Last-In First-Out)



© S. Prasitjutrakul 2013

การเพิ่ม/ลบข้อมูลในกองซ้อน

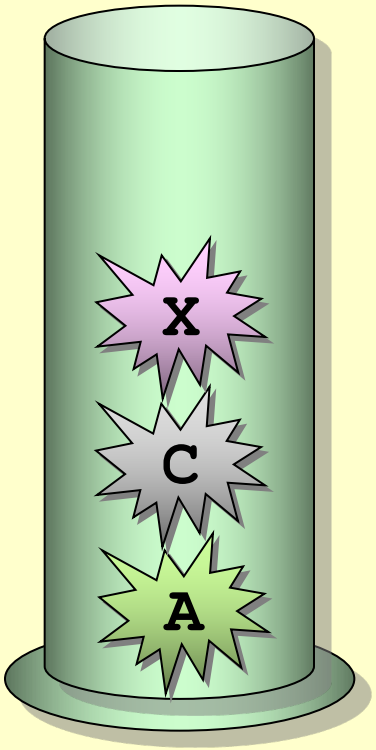
- ข้อมูล เข้าหลัง ออกก่อน (Last-In First-Out)



© S. Prasitjutrakul 2013

การเพิ่ม/ลบข้อมูลในกองซ้อน

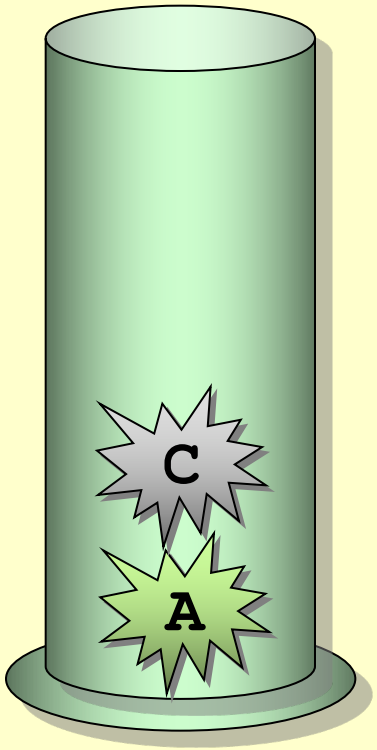
- ข้อมูล เข้าหลัง ออกก่อน (Last-In First-Out)



© S. Prasitjutrakul 2013

การเพิ่ม/ลบข้อมูลในกองซ้อน

- ข้อมูล เข้าหลัง ออกก่อน (Last-In First-Out)



© S. Prasitjutrakul 2013

กองซ้อน : stack

```
bool        empty();
unsigned    size();
T           top();
void        push(T element);
void        pop();
```


© S. Prasitjutrakul 2013

ตัวอย่างการใช้งาน Stack

- การตรวจสอบโครงสร้างแบบซ้อนกัน
  เช่น การใส่วงเล็บ ( ) { } [ ] ...
- การจัดเก็บตัวแปรและการทำ function calls
- การประมวลผลนิพจน์ทางคณิตศาสตร์
- การทำ undo/redo
- การค้นคำตอบแบบ depth-first search
- ...

© S. Prasitjutrakul 2013

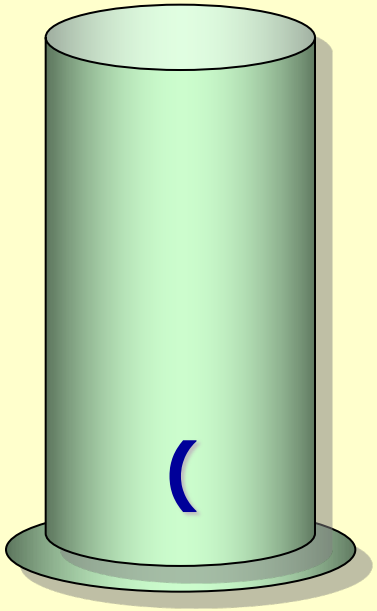
การตรวจสอบการใส่วงเล็บ

- ถูก : ({ ( ) [{ }] })
- ผิด : เปิดปิดไม่ตรงกัน ({])
- ผิด : มีปิดมากไป ({ ( ) })) }
- ผิด : มีเปิดมากไป ({ ( ) }
- วิธีทำ
  - อ่านมาทีละตัว
  - ถ้าเป็นวงเล็บเปิด ให้ push ลง stack
  - ถ้าเป็นวงเล็บปิด ให้ pop จาก stack มาตรวจสอบว่าเป็นวงเล็บเปิดที่ตรงกันวงเล็บปิดที่พบหรือไม่
  - เมื่อใดอยาก pop ถ้า isEmpty แสดงว่า ปิดมีมากไป
  - เมื่ออ่านเสร็จหมด stack ยังมีข้อมูล แสดงว่า เปิดมีมากไป

© S. Prasitjutrakul 2013

ตัวอย่าง ๑

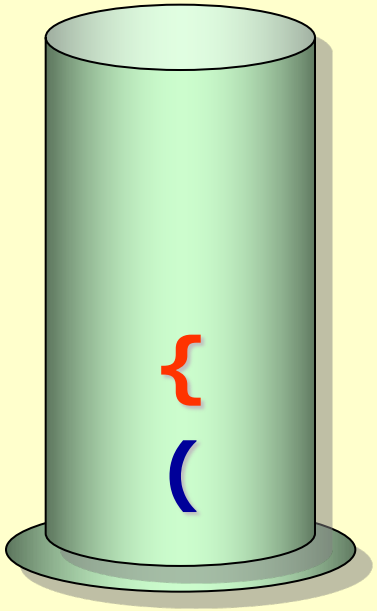
- ({ ( ) [{ }] })



© S. Prasitjutrakul 2013

ตัวอย่าง ๑

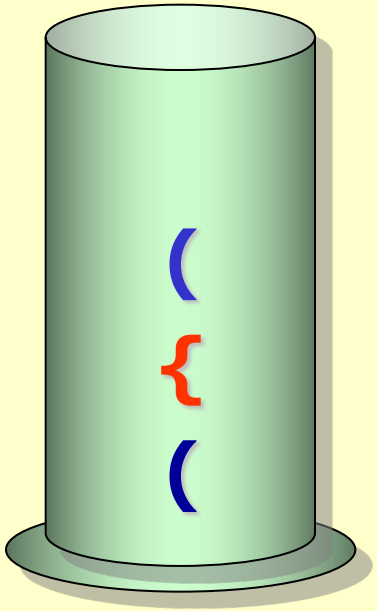
- ({ ( ) [{ }] })



© S. Prasitjutrakul 2013

ตัวอย่าง ๑

- ({ ( ) [{ }] })

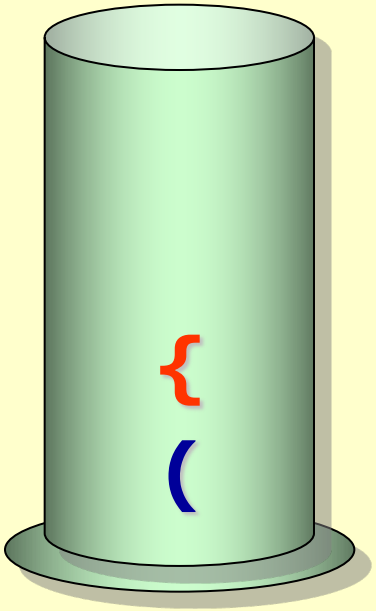


© S. Prasitjutrakul 2013

ตัวอย่าง ๑

- ({ ( ) [{ }] })

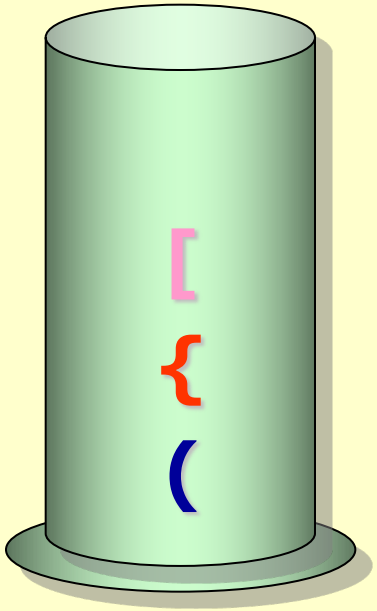




© S. Prasitjutrakul 2013

ตัวอย่าง ๑

- ({ ( ) [{ }] })

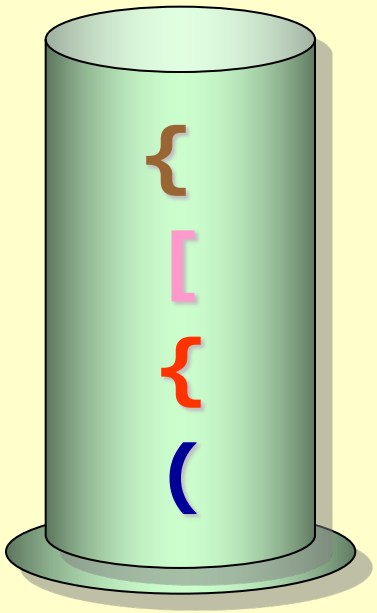


© S. Prasitjutrakul 2013

ตัวอย่าง ๑

- ({ ( ) [{ }] })

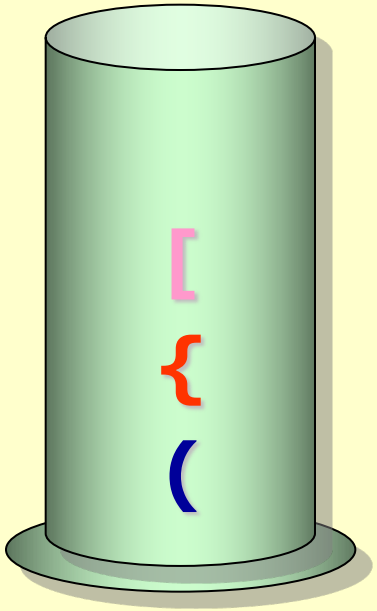




© S. Prasitjutrakul 2013

ตัวอย่าง ๑

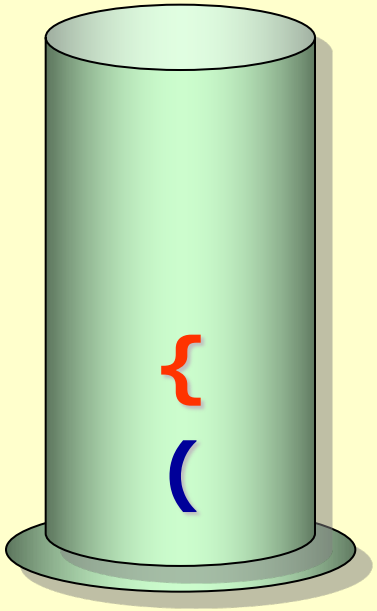
- ({ ( ) [{ }] })



© S. Prasitjutrakul 2013

ตัวอย่าง ๑

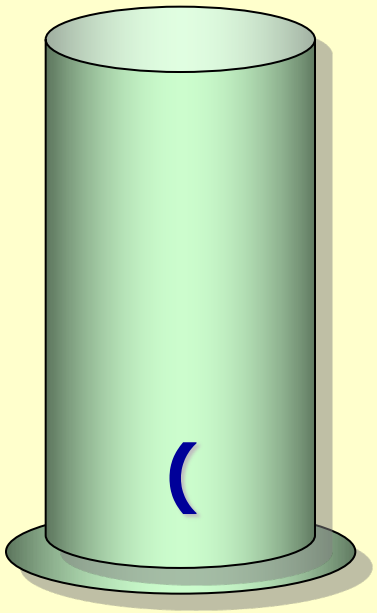
- ({ ( ) [{ }] })



© S. Prasitjutrakul 2013

ตัวอย่าง ๑

- ({ ( ) [{ }] })

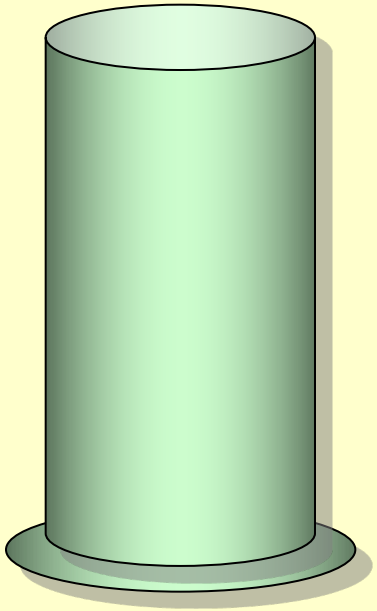


© S. Prasitjutrakul 2013

ตัวอย่าง ๑

- ({ ( ) [{ }] })

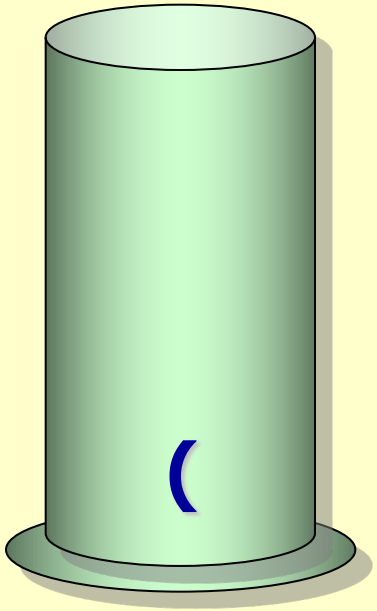
ถูกต้อง



© S. Prasitjutrakul 2013

ตัวอย่าง ๒

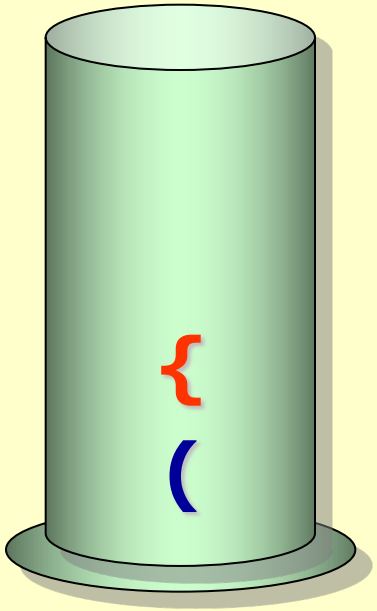
- ({ ( ) [{)] })



© S. Prasitjutrakul 2013

ตัวอย่าง ๒

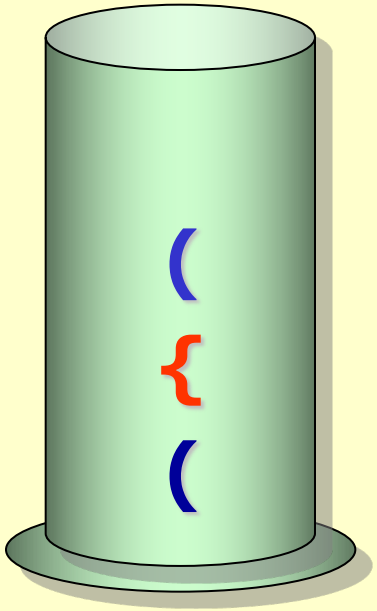
- ({ ( ) [{)] })



© S. Prasitjutrakul 2013

ตัวอย่าง ๒

- ({ ( ) [{)] })

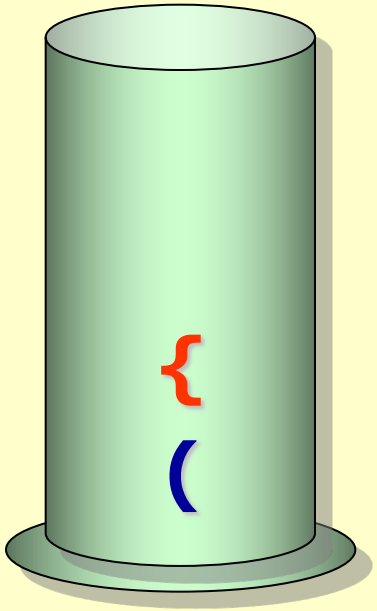


© S. Prasitjutrakul 2013

ตัวอย่าง ๒

- ({ ( ) [{)] })



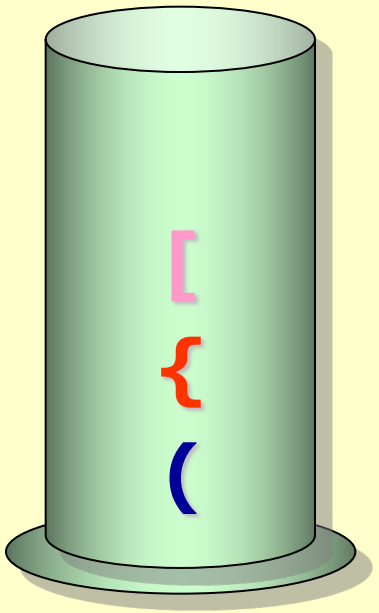


© S. Prasitjutrakul 2013

ตัวอย่าง ๒

- ({ ( ) [{)] })



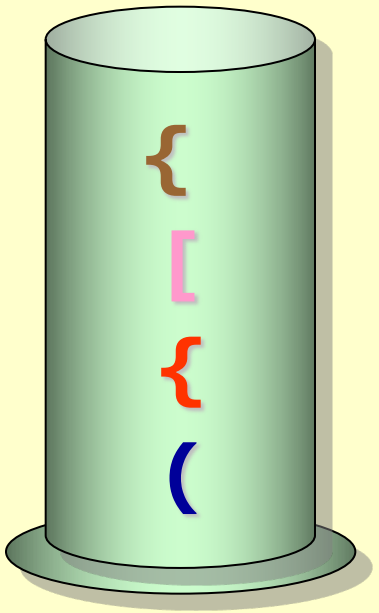


© S. Prasitjutrakul 2013

ตัวอย่าง ๒

- ({ ( ) [{)] })





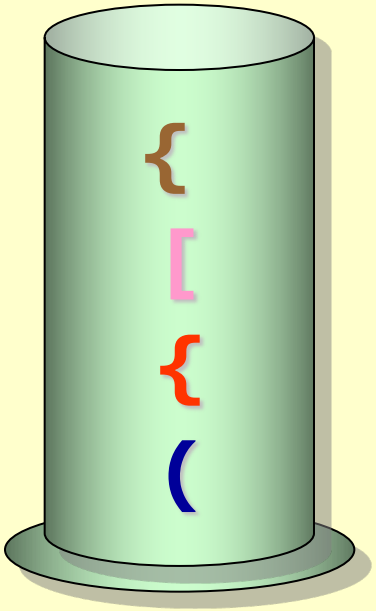
© S. Prasitjutrakul 2013

ตัวอย่าง ๒

- ({ ( ) [{)] })



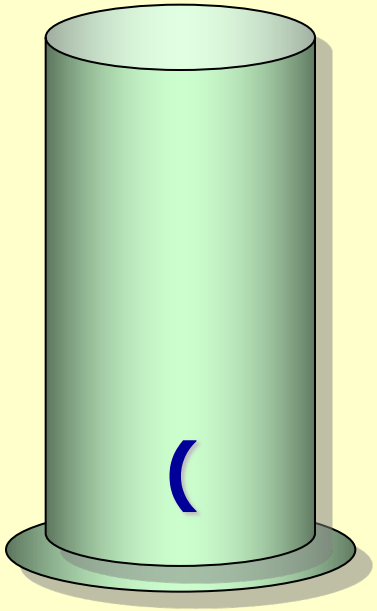
วงเล็บปิด
ไม่ตรงกับเปิด
ผิด !!



© S. Prasitjutrakul 2013

ตัวอย่าง ๓

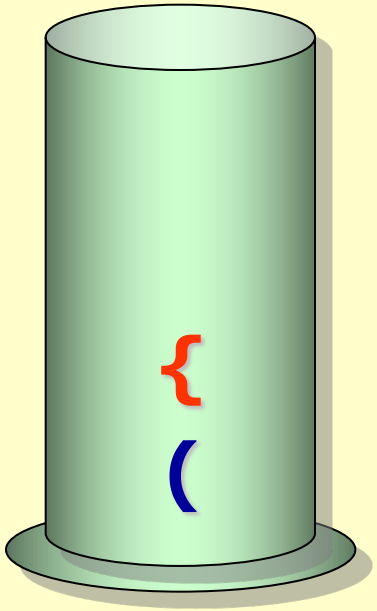
- ({ ( ) })]



© S. Prasitjutrakul 2013

ตัวอย่าง ๓

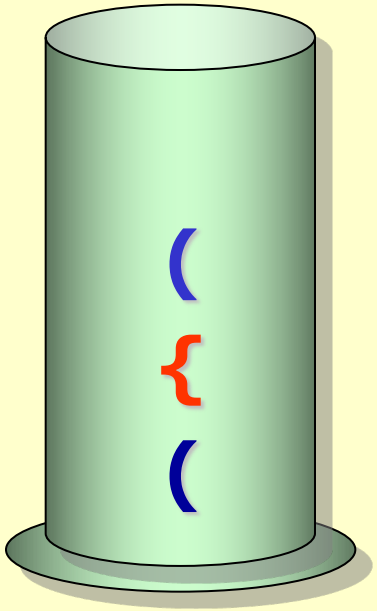
- ({ ( ) })]



© S. Prasitjutrakul 2013

ตัวอย่าง ๓

- ({ ( ) })]

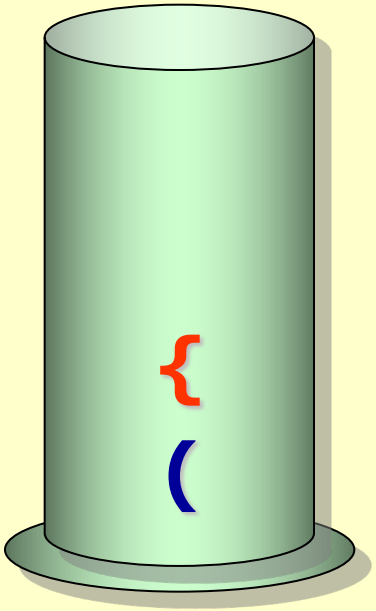


© S. Prasitjutrakul 2013

ตัวอย่าง ๓

- ({ ( ) })]



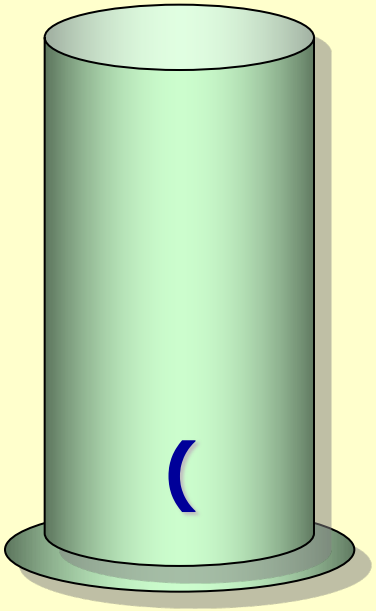


© S. Prasitjutrakul 2013

ตัวอย่าง ๓

- ({ ( ) })]





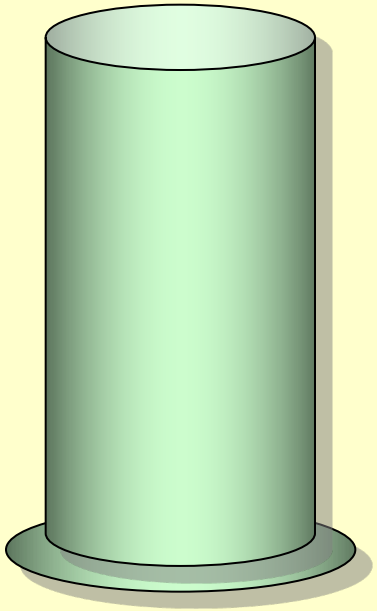
© S. Prasitjutrakul 2013

ตัวอย่าง ๓

- ({ ( ) })]



ยังไม่หมด
แต่ stack ว่าง
ผิด !!



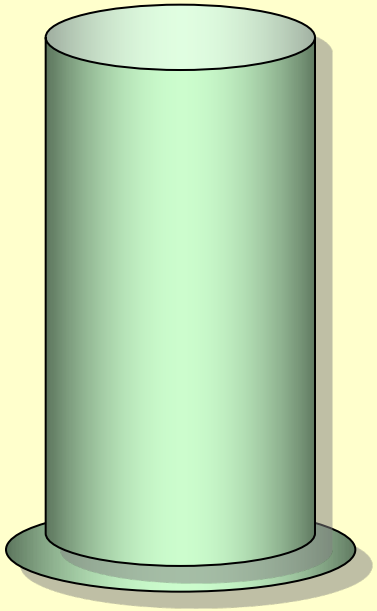
© S. Prasitjutrakul 2013

ตัวอย่าง ๓

- ({ ( ) })]



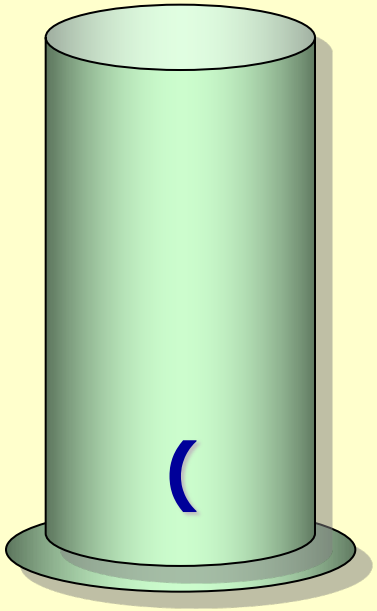
ยังไม่หมด
แต่ stack ว่าง
ผิด !!



© S. Prasitjutrakul 2013

ตัวอย่าง ๔

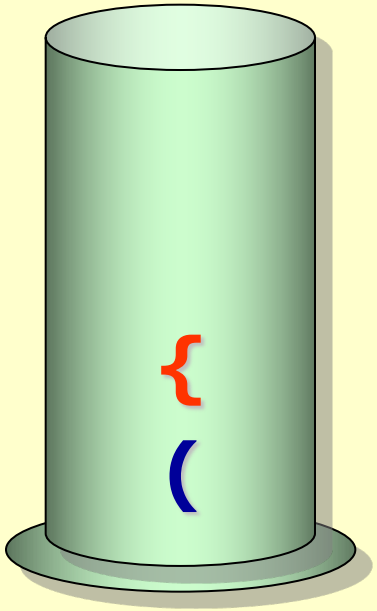
- ({ ( ) [{ }



© S. Prasitjutrakul 2013

ตัวอย่าง ๔

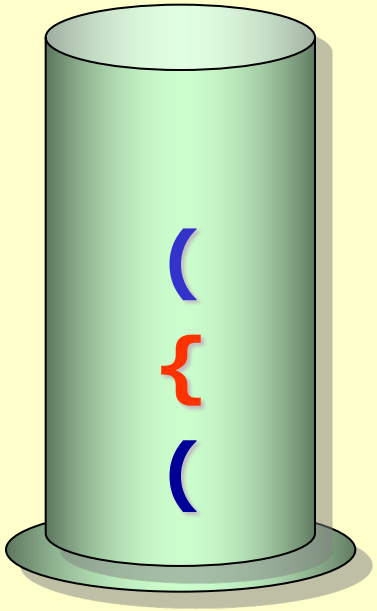
- ({ ( ) [{ }



© S. Prasitjutrakul 2013

ตัวอย่าง ๔

- ({ ( ) [{ }

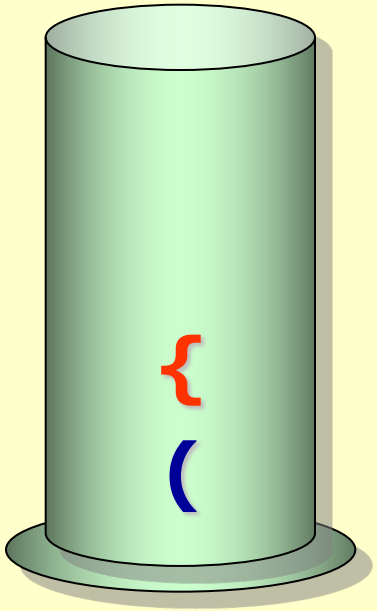


© S. Prasitjutrakul 2013

ตัวอย่าง ๔

- ({ ( ) [{ }



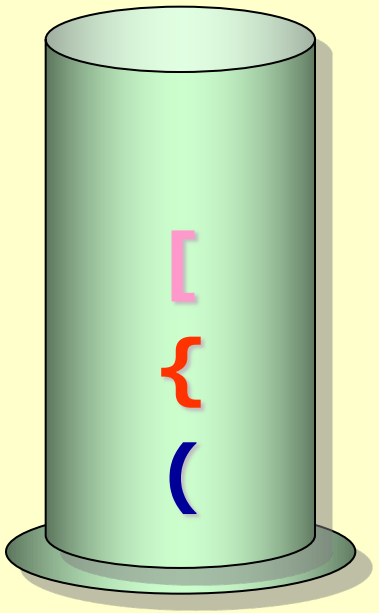


© S. Prasitjutrakul 2013

ตัวอย่าง ๔

- ({ ( ) [{ }



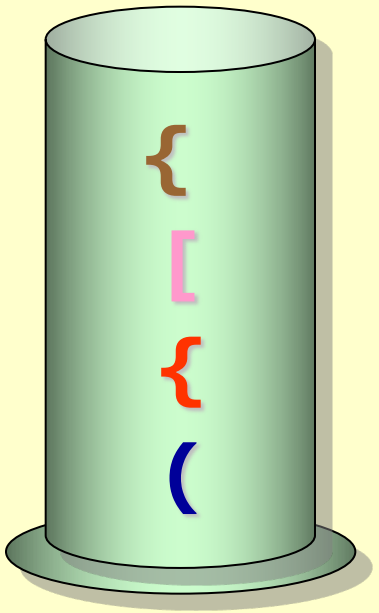


© S. Prasitjutrakul 2013

ตัวอย่าง ๔

- ({ ( ) [{ }



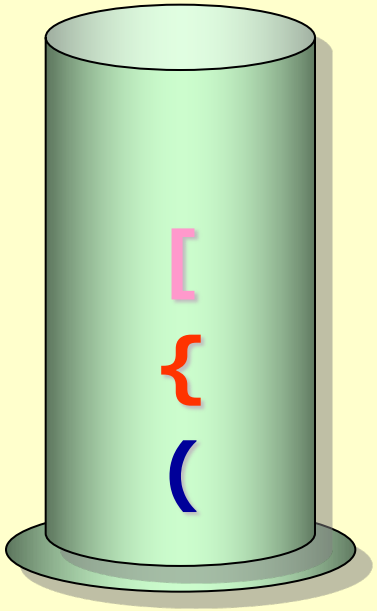


© S. Prasitjutrakul 2013

ตัวอย่าง ๔

- ({ ( ) [{ }

หมดแล้ว
แต่ stack ไม่ว่าง
ผิด !!



© S. Prasitjutrakul 2013

การใช้กองซ้อนภายใน java virtual machine

- ตัว jvm ใช้กองซ้อน (java stack) ในการเก็บ
  - สถานะของการเรียกเมท็อด
  - พารามิเตอร์ และ local variables
  - ที่เก็บชั่วคราวเพื่อการคำนวณ (operand stack)

```
public class Jeng3 {
  public static void main(String[] a) {
    main(args);
  }
}
```

```
Exception in thread "main" java.lang.StackOverflowError
        at Jeng3.main(Jeng3.java:3)
```

```
int main() {
  main();
}
```

```
Process terminated with status -1073741571
```

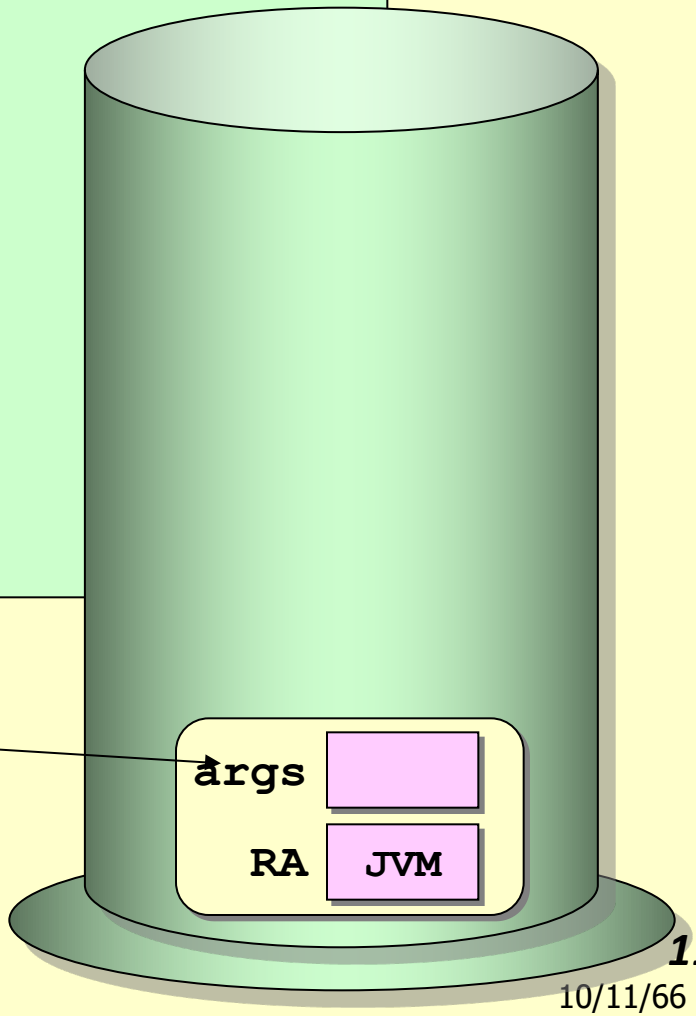

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:   public static void main(String[] args) {
03:     a(3, 2);
04:     b(5);
05:   }
06:   static void a(int x, int y) {
07:     int z = x/y;
08:     b(z);
09:   }
10:   static void b(int x) {
11:     ++x;
12:   }
13:}
```

กรอบกองซ้อน (Stack Frame)

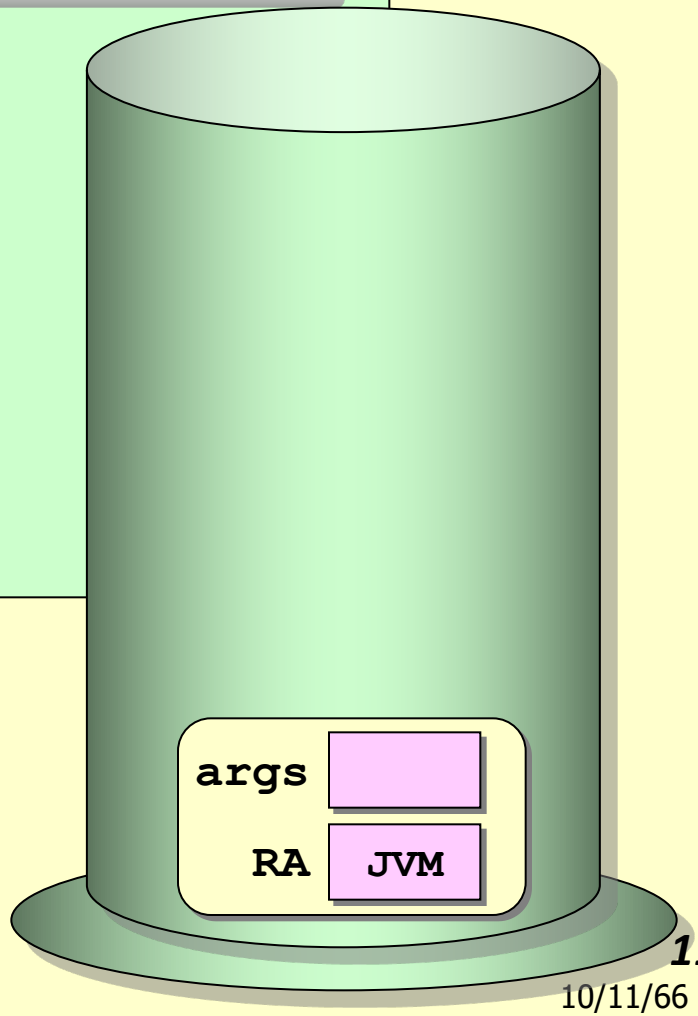
parameters & local variables



© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:   public static void main(String[] args) {
03:     a(3, 2);
04:     b(5);
05:   }
06:   static void a(int x, int y) {
07:     int z = x/y;
08:     b(z);
09:   }
10:   static void b(int x) {
11:     ++x;
12:   }
13:}
```

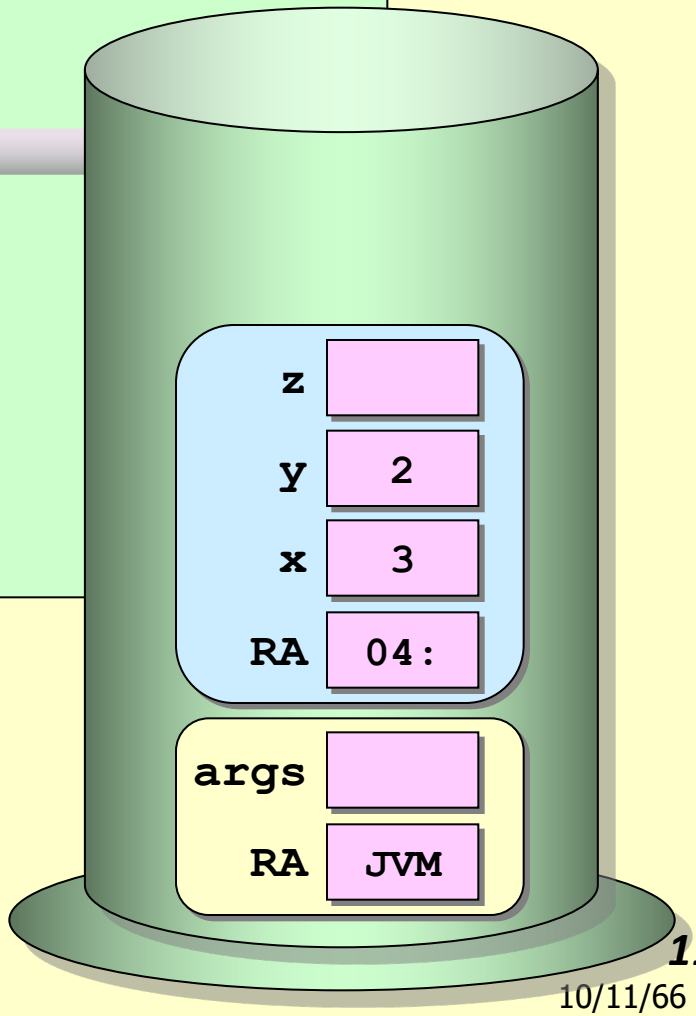


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:   public static void main(String[] args) {
03:     a(3, 2);
04:     b(5);
05:   }
06:   static void a(int x, int y) {
07:     int z = x/y;
08:     b(z);
09:   }
10:   static void b(int x) {
11:     ++x;
12:   }
13:}
```

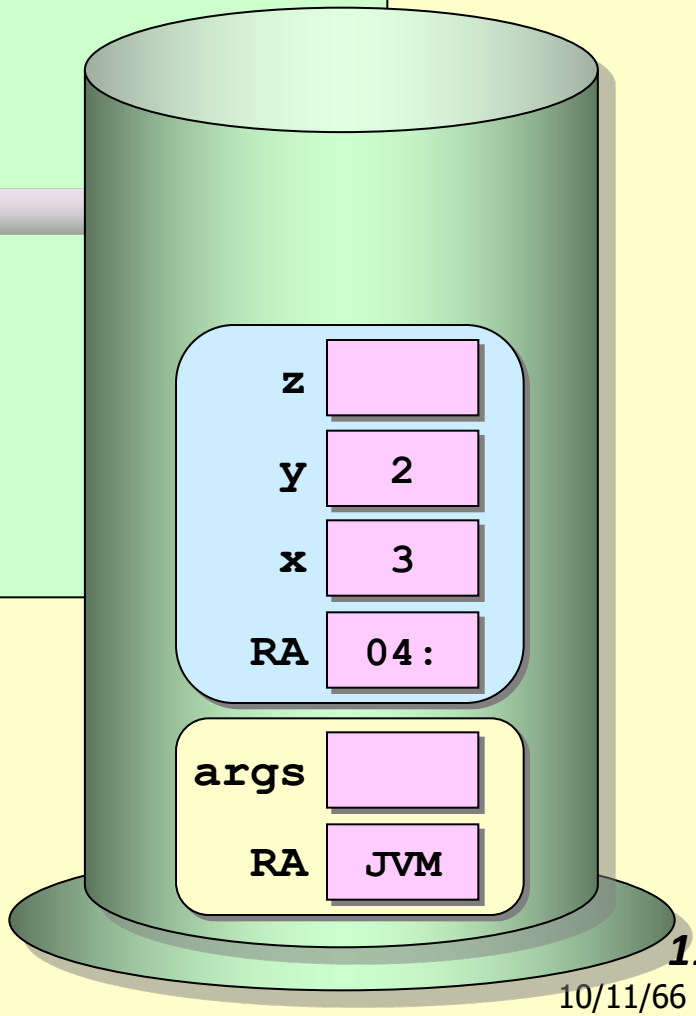


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```

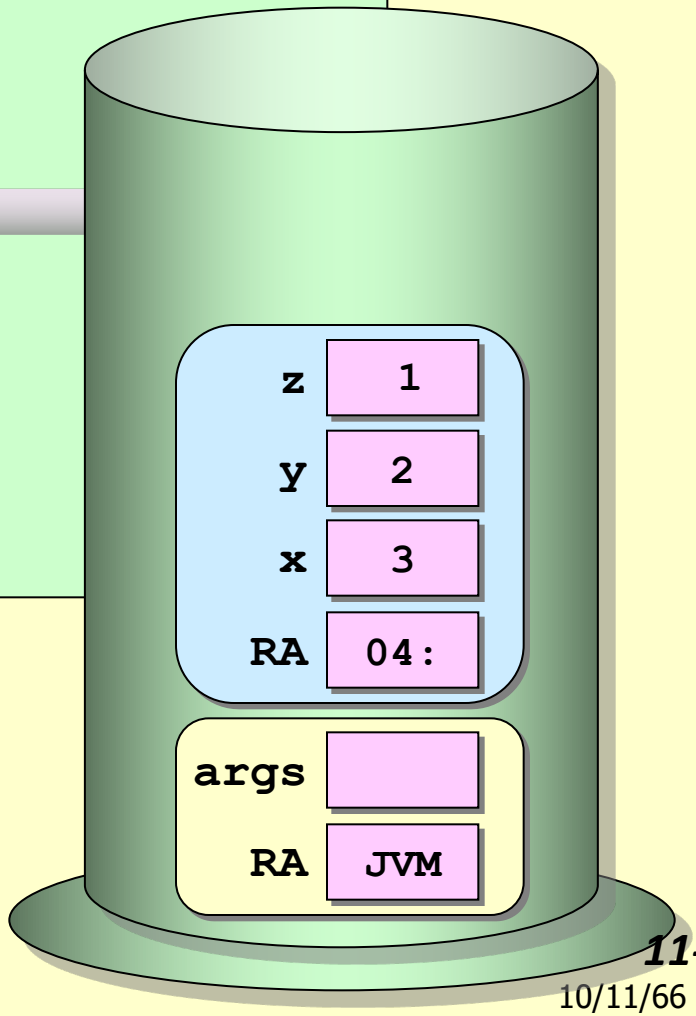


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```

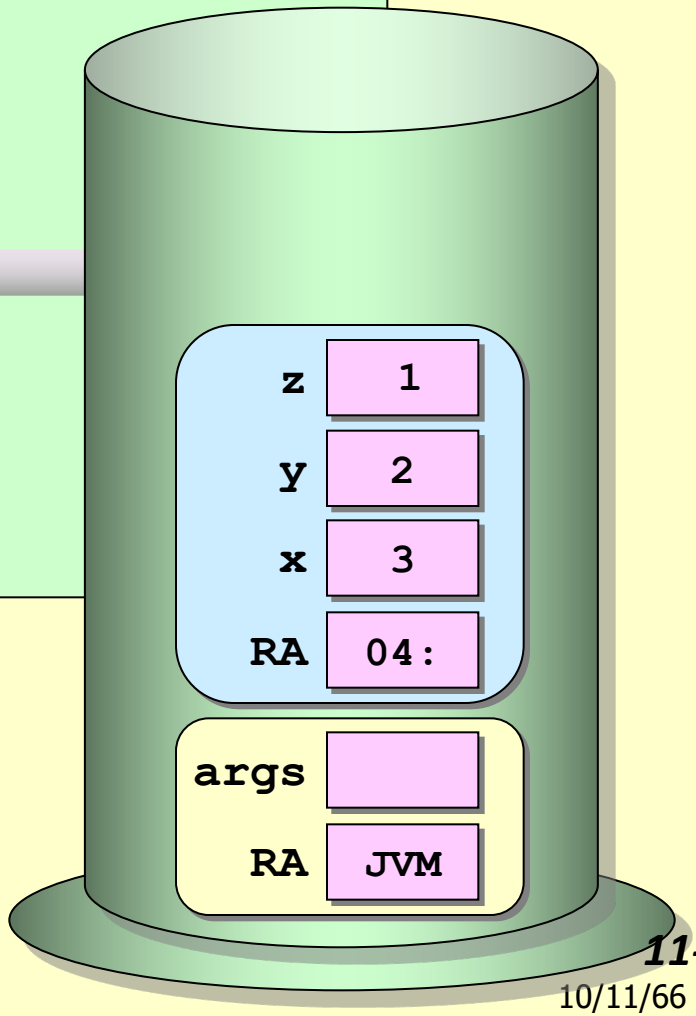


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:   public static void main(String[] args) {
03:     a(3, 2);
04:     b(5);
05:   }
06:   static void a(int x, int y) {
07:     int z = x/y;
08:     b(z);
09:   }
10:   static void b(int x) {
11:     ++x;
12:   }
13:}
```

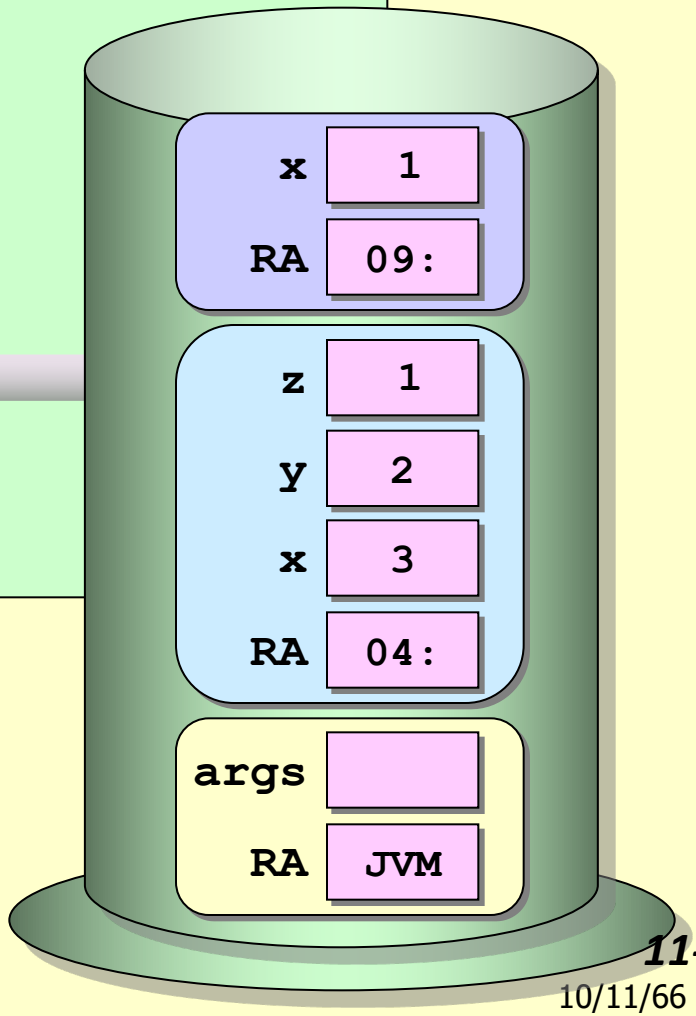


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```

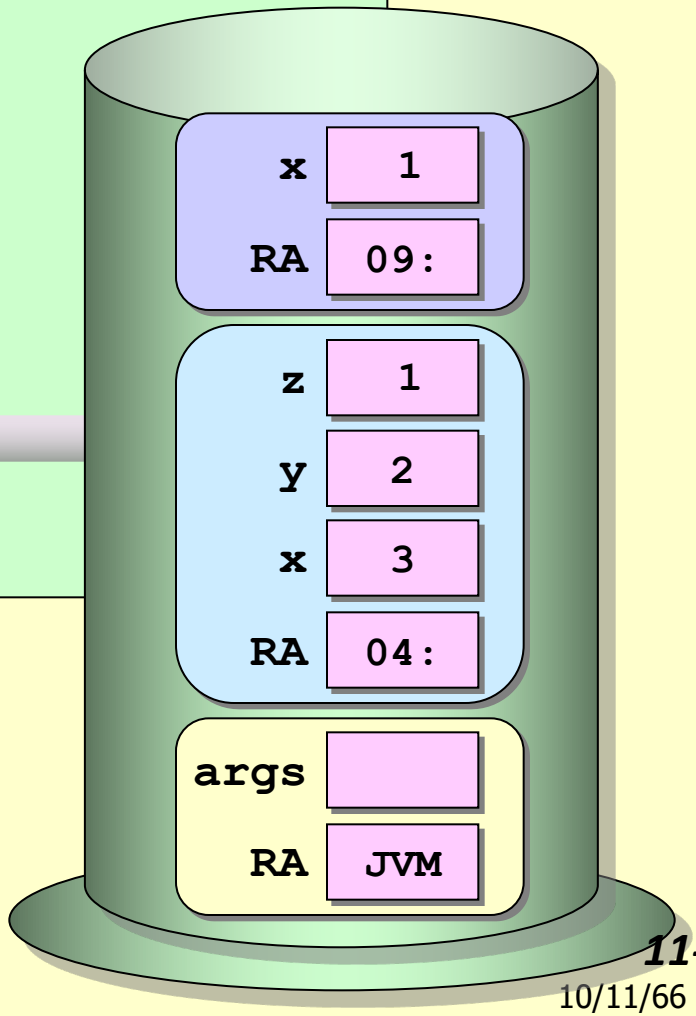


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```

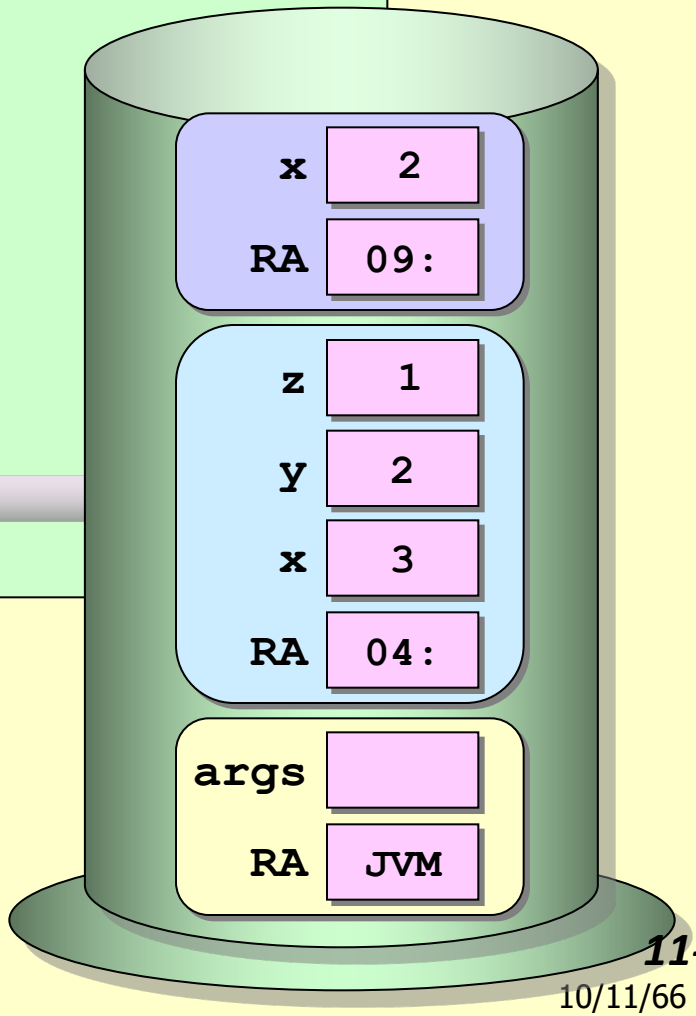


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```

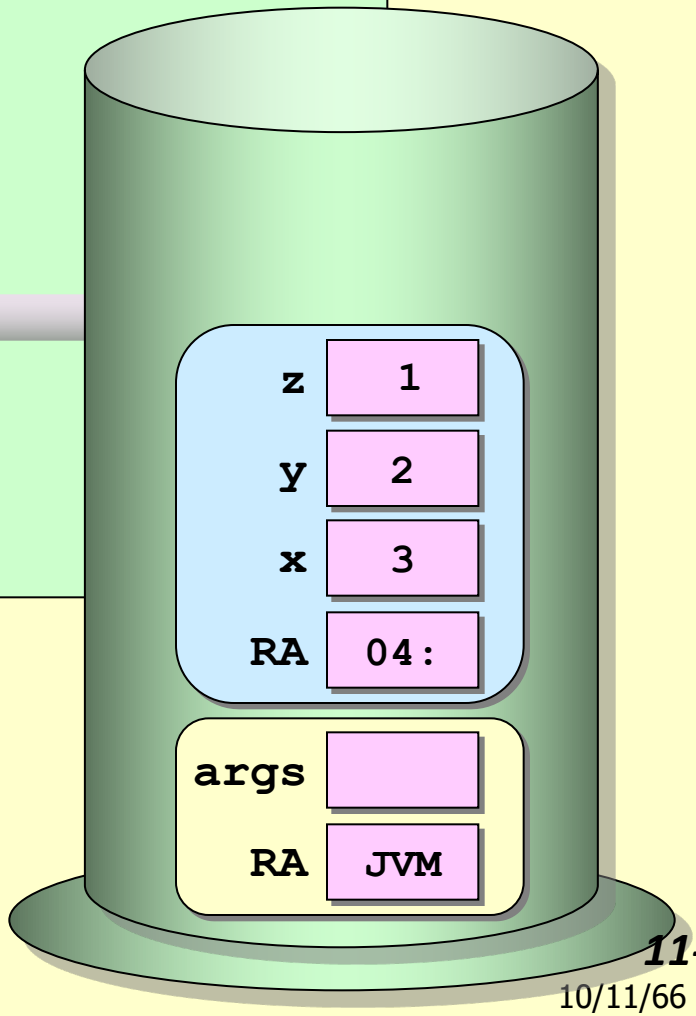


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```



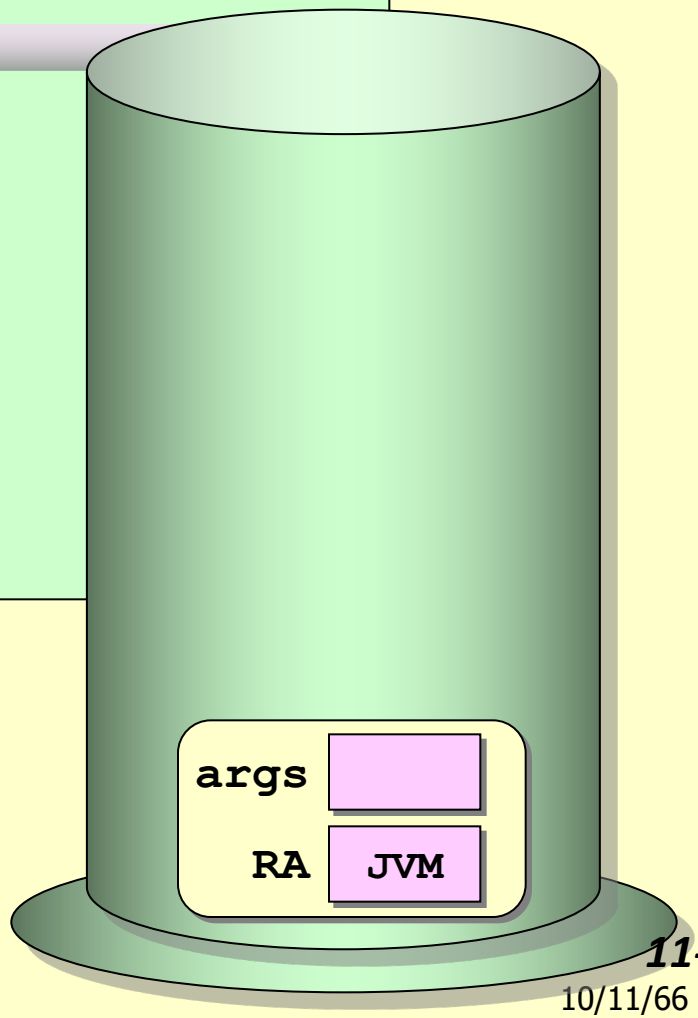
กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:   public static void main(String[] args) {
03:     a(3, 2);
04:     b(5);
05:   }
06:   static void a(int x, int y) {
07:     int z = x/y;
08:     b(z);
09:   }
10:   static void b(int x) {
11:     ++x;
12:   }
13:}
```

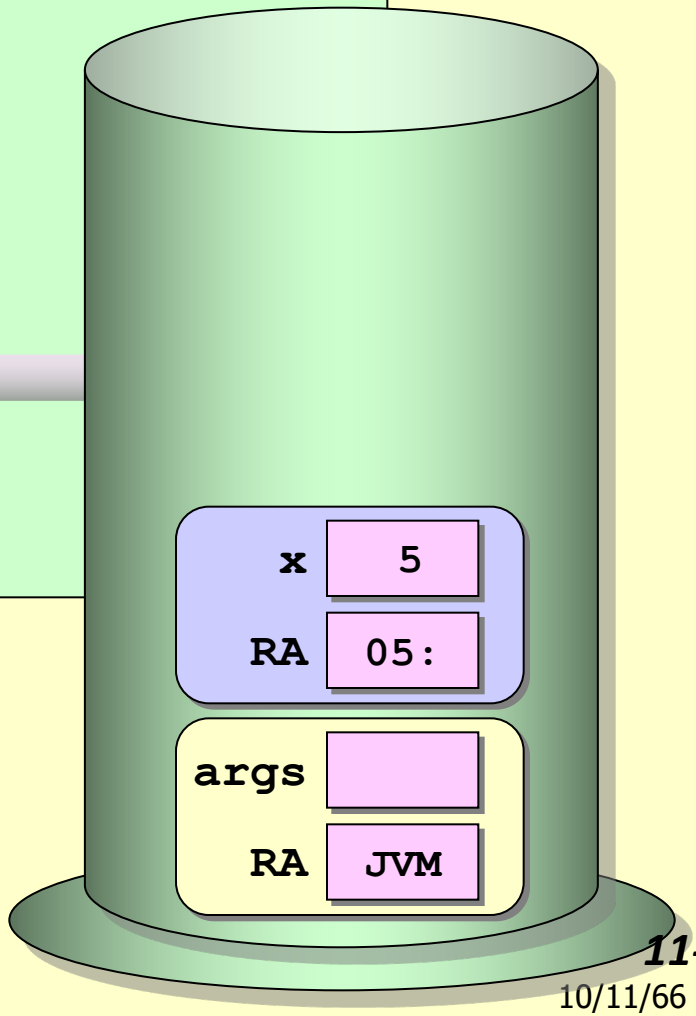
กรอบกองซ้อน
(Stack Frame)



© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```

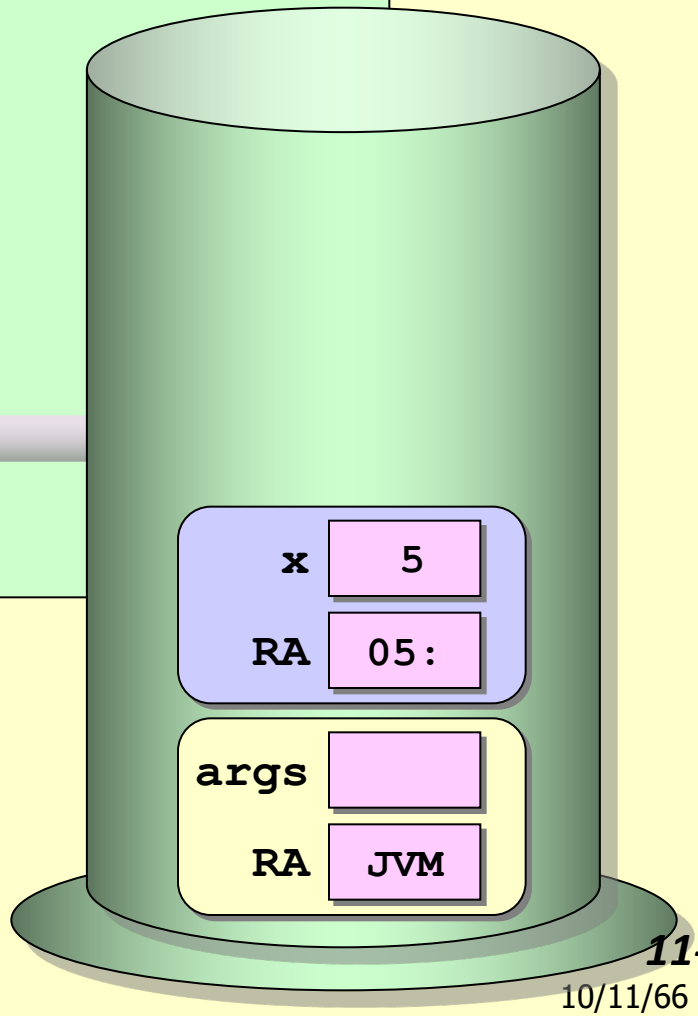


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```

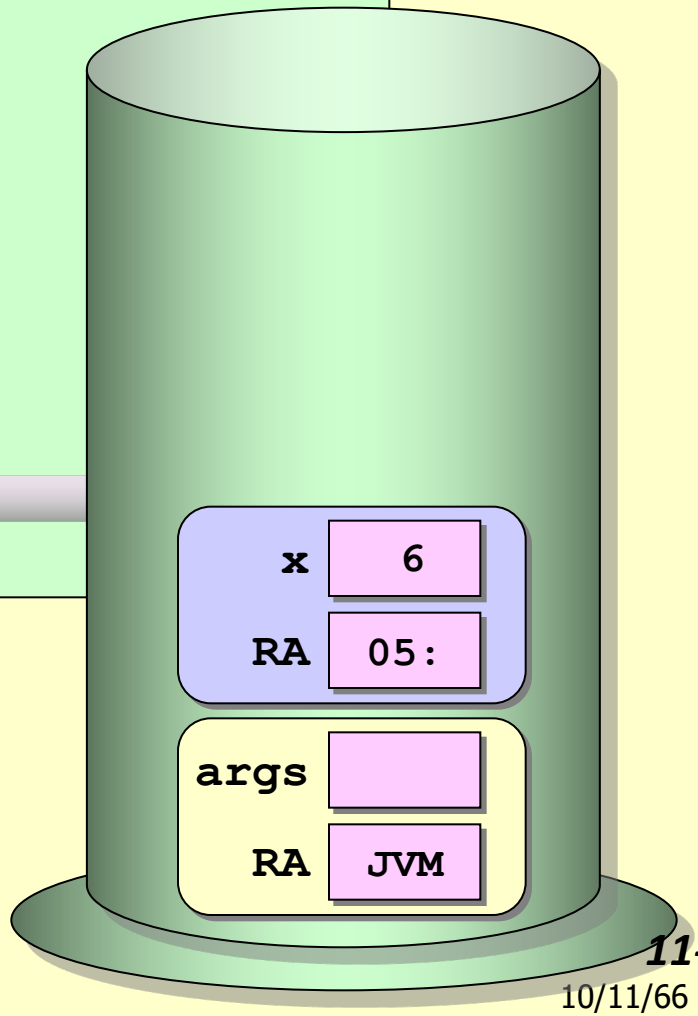


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```

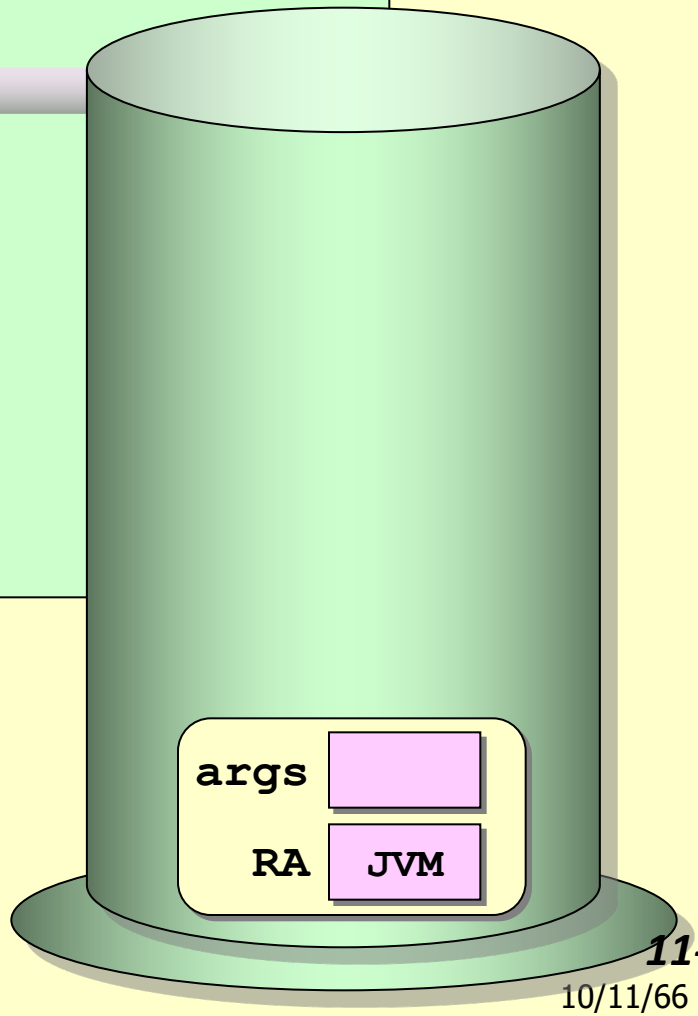


กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```



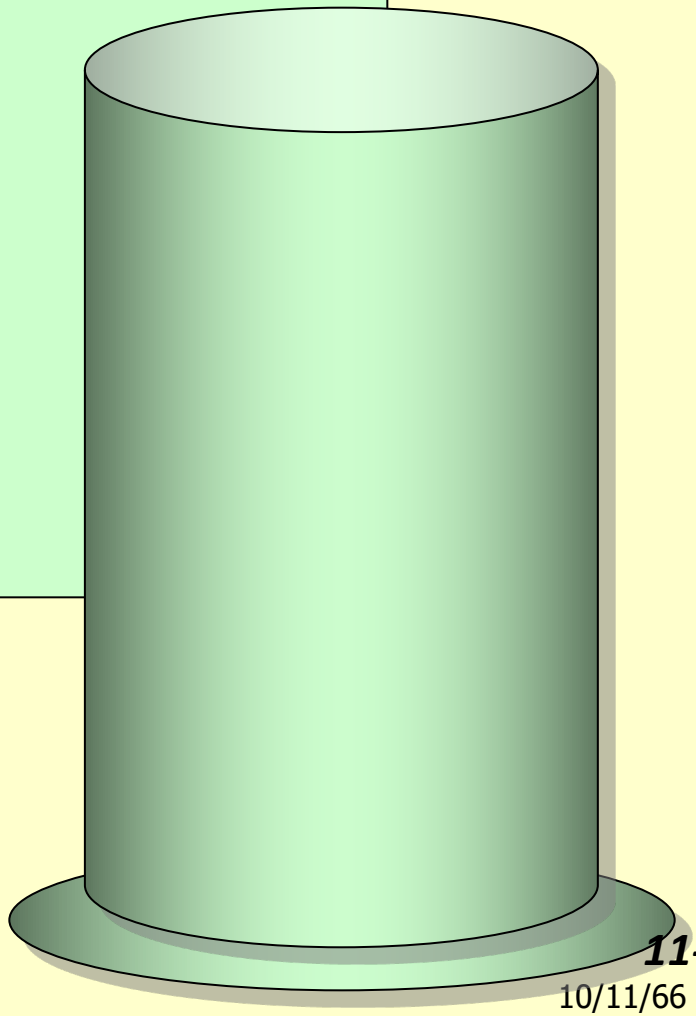
กรอบกองซ้อน
(Stack Frame)

© S. Prasitjutrakul 2013

Java Stack

```
01:public class StackFrame {
02:  public static void main(String[] args) {
03:    a(3, 2);
04:    b(5);
05:  }
06:  static void a(int x, int y) {
07:    int z = x/y;
08:    b(z);
09:  }
10:  static void b(int x) {
11:    ++x;
12:  }
13:}
```

กรอบกองซ้อน
(Stack Frame)



© S. Prasitjutrakul 2013

นิพจน์ Infix และ Postfix

- infix (เติมกลาง)
  - a + b * c / d – 2, (a + b) * c / (d – 2)
  - ต้องกำหนดลำดับการทำก่อนหลังของ operators
  - ใช้วงเล็บช่วย
- postfix (เติมหลัง)
  - a b c * d / + 2 – , a b + c * d 2 - /
  - ลำดับการทำงานของ operators คือจาก ซ้ายไปขวา
  - ไม่จำเป็นต้องมีวงเล็บ

```
1   2   +   3   *   4   1   -   /
        3   3   *   4   1   -   /
```

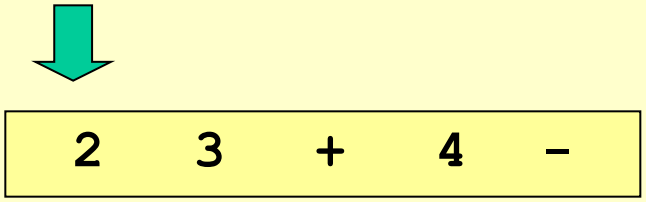

© S. Prasitjutrakul 2013

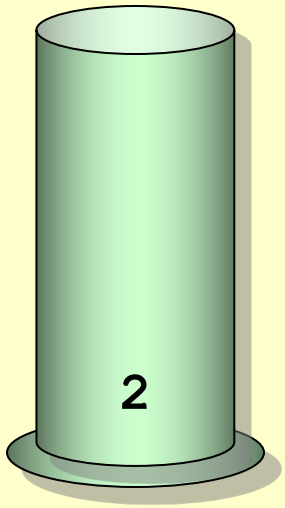
การใช้กองซ้อนคำนวณค่าของนิพจน์เติมหลัง

- 2 3 + 4 5 – 6 * + มีค่าเท่าไร ?
- สามารถใช้ stack ช่วยหาค่าของนิพจน์ postfix
- วิธีทำ
  - ดูทีละตัวใน postfix จากซ้ายไปขวา
  - ถ้าเป็น operand ให้ push ของ stack
  - ถ้าเป็น operator ให้ pop operands จาก stack ตามที่ operator ต้องการมาประมวลผล แล้ว push ผลลัพธ์
  - ทำเสร็จ คำตอบจะอยู่ที่ top of stack

© S. Prasitjutrakul 2013

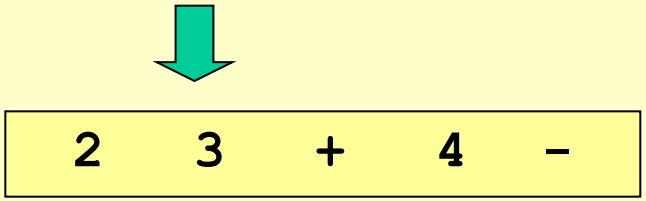
ตัวอย่าง

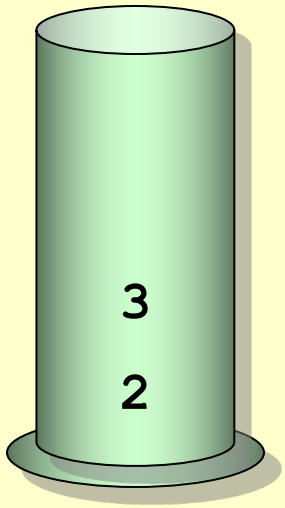




© S. Prasitjutrakul 2013

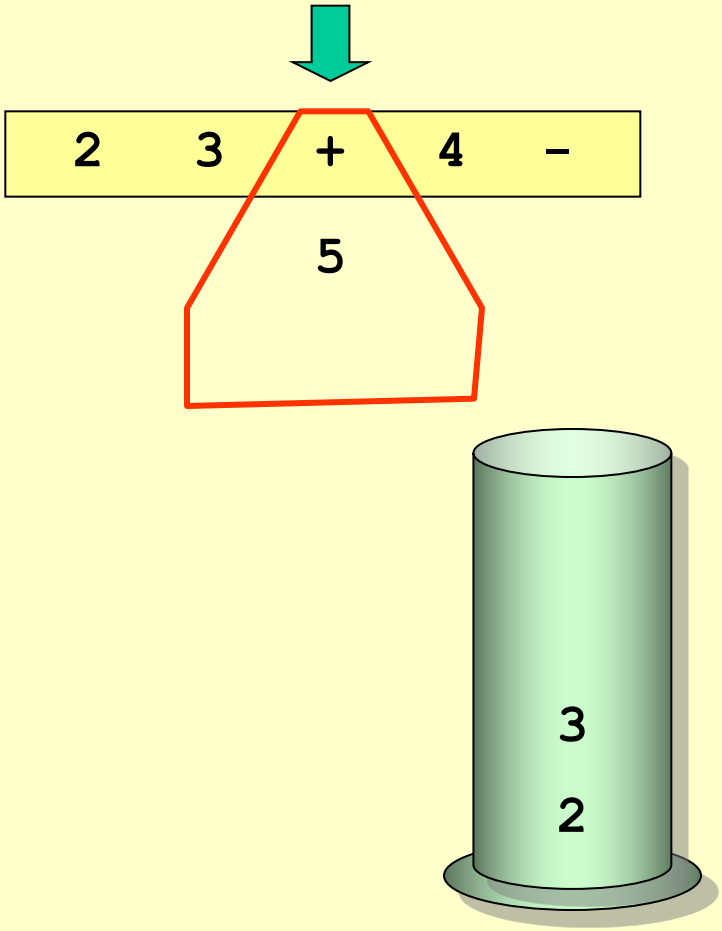
ตัวอย่าง





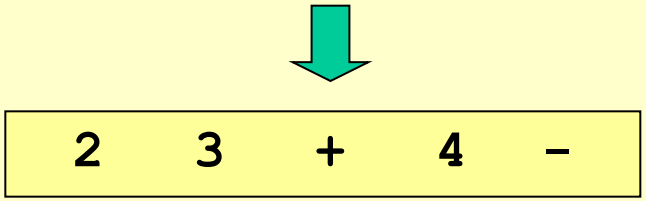
© S. Prasitjutrakul 2013

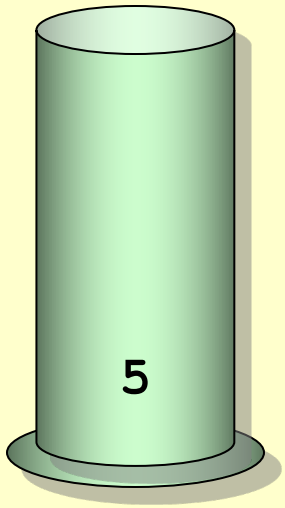
ตัวอย่าง



© S. Prasitjutrakul 2013

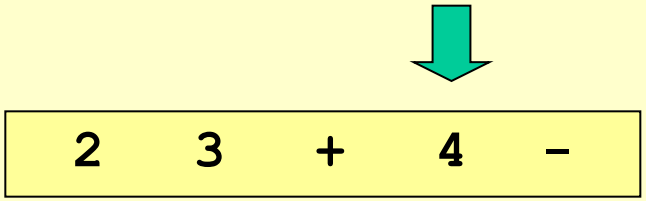
ตัวอย่าง

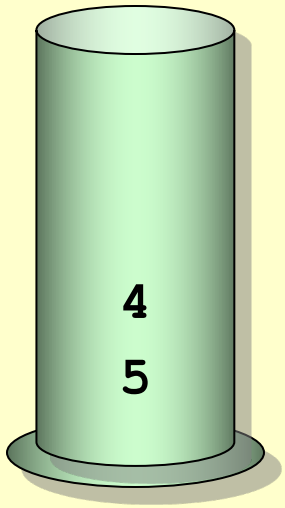




© S. Prasitjutrakul 2013

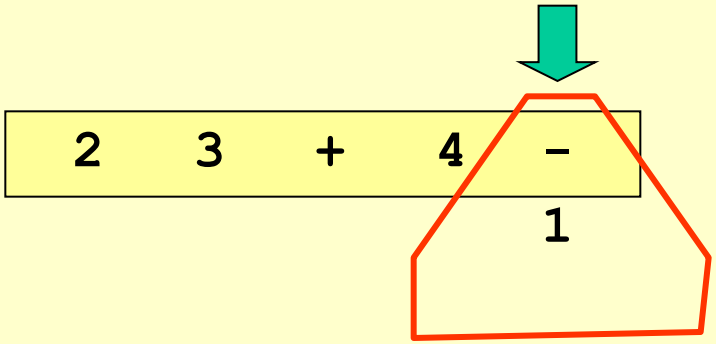
ตัวอย่าง

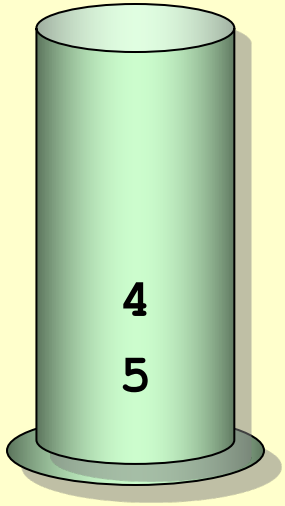




© S. Prasitjutrakul 2013

ตัวอย่าง



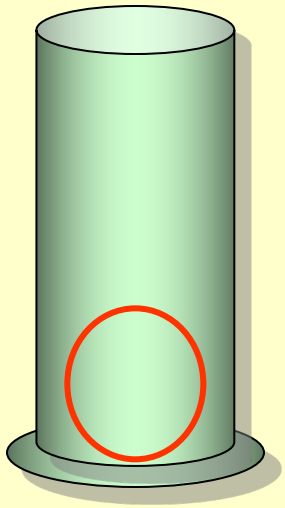


© S. Prasitjutrakul 2013

ตัวอย่าง

2	3	+	4	-

1



ผลลัพธ์อยู่บนกองซ้อน

© S. Prasitjutrakul 2013

jvm เป็น stack machine

`int a = 1;`	`iconst_1` `istore_1`
`int b = 2;`	`iconst_2` `istore_2`
`int c = 3;`	`iconst_3` `istore_3`

`int d = a + b * (c + a);`	`iload_1`
	`iload_2`
	`iload_3`
	`iload_1`
	`iadd`
	`imul`
	`iadd`
	`istore_4`

infix

postfix

© S. Prasitjutrakul 2013

การใช้กองซ้อนช่วยแปลง infix เป็น postfix

- input : infix expression
- output : postfix expression
- ขั้นตอนการทำงาน
  - ดูแต่ละตัวใน infix
  - ถ้าเป็น operand นำไปต่อท้าย output
  - ถ้าเป็น operator
    - อาจ pop operator ออกไปต่อท้าย output
    - push operator ลงกองซ้อน
  - เมื่อดู infix ครบตัวแล้ว
    - ให้ pop operators ทุกตัวออกไปต่อท้าย output

© S. Prasitjutrakul 2013

infix -> postfix

```
string infix2postfix(string &infix) {
  int n = infix.length();
  string postfix = "";
  stack<char> s;
  for (int i=0; i<n; i++) {
    char token = infix[i];
    if ( token เป็น operand ) {
      postfix += token;
    } else {
      while(  ????  ) {
        postfix += s.top();
        s.pop();
      }
      s.push(token);
    }
  }
  while(!s.empty()) {postfix += s.top(); s.pop();}
  return postfix;
}
```

ดูทีละตัว

ถ้าเป็น operand, เพิ่มต่อในผลลัพธ์

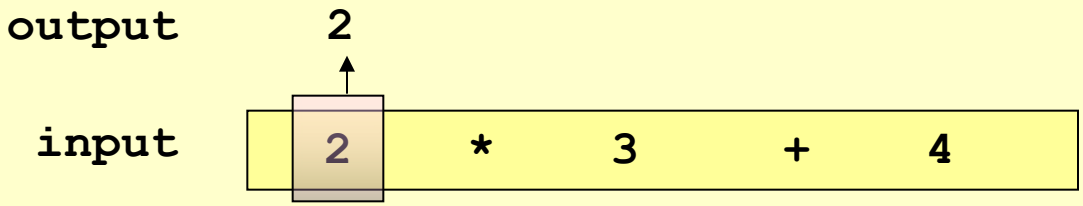
ถ้าเป็น operator

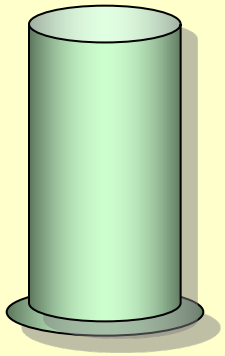
อาจ pop operators ใน stack ออกเป็นผลลัพธ์

ตามด้วยการ push operator ตัวใหม่

© S. Prasitjutrakul 2013

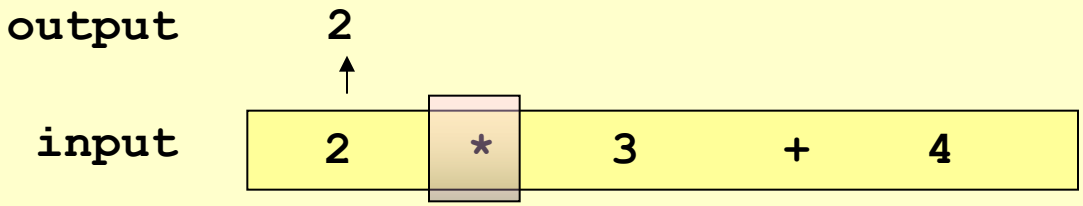
ความสำคัญของ operators

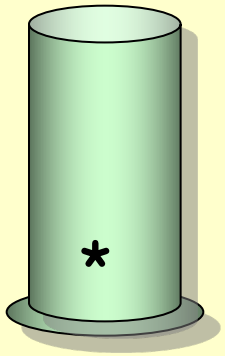




© S. Prasitjutrakul 2013

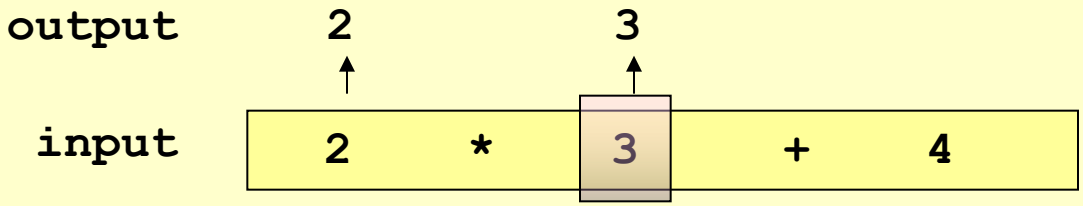
ความสำคัญของ operators

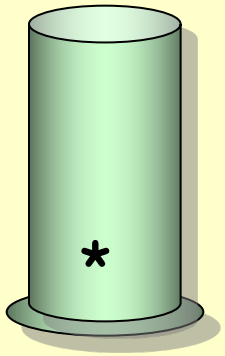




© S. Prasitjutrakul 2013

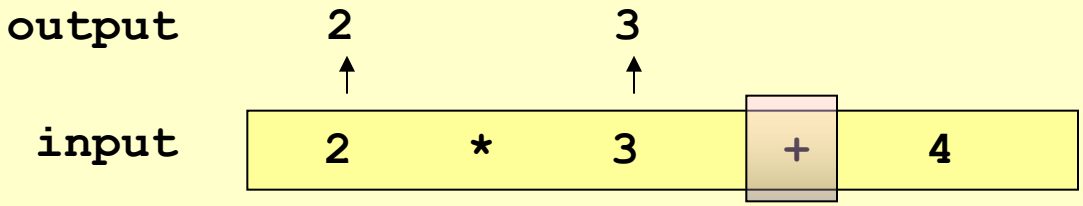
ความสำคัญของ operators

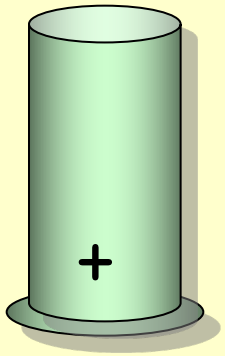




© S. Prasitjutrakul 2013

ความสำคัญของ operators



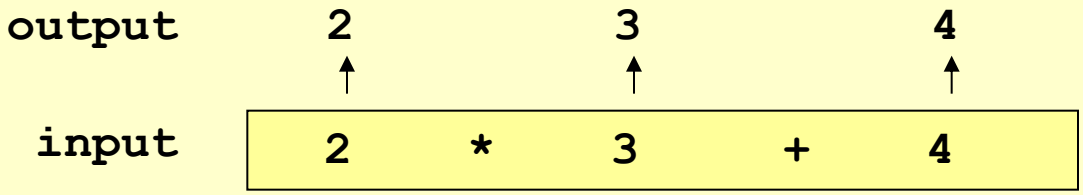


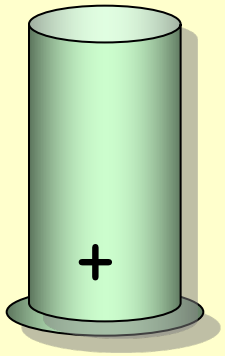
* มาก่อน และสำคัญกว่า +

pop ออก ถ้า operator บนกองซ้อนสำคัญกว่า operator ใหม่

© S. Prasitjutrakul 2013

ความสำคัญของ operators



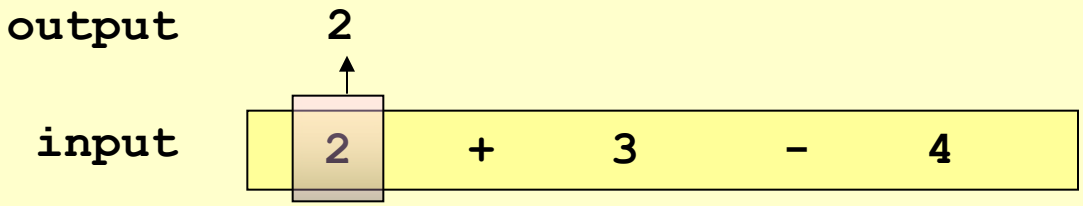


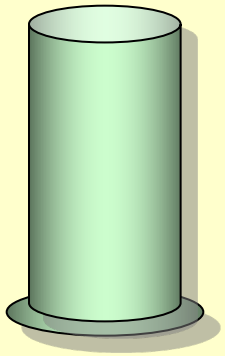
* มาก่อน และสำคัญกว่า +

pop ออก ถ้า operator บนกองซ้อนสำคัญกว่า operator ใหม่

© S. Prasitjutrakul 2013

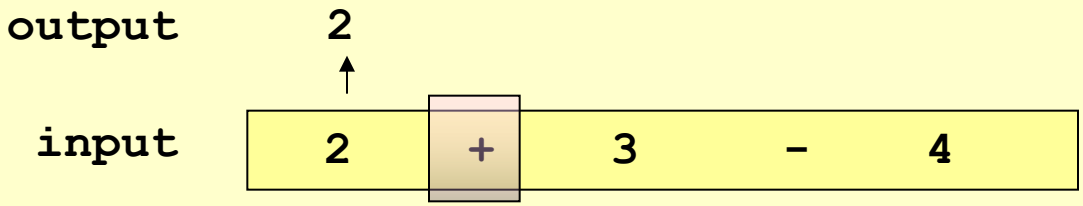
ความสำคัญของ operators

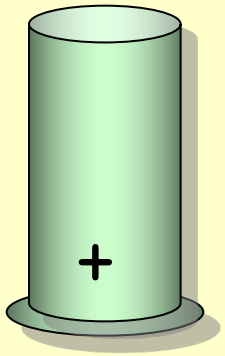




© S. Prasitjutrakul 2013

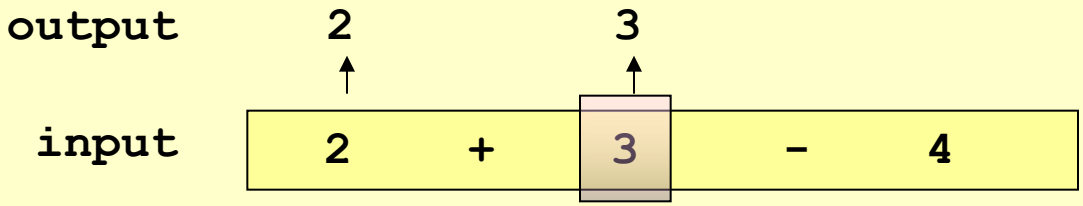
ความสำคัญของ operators

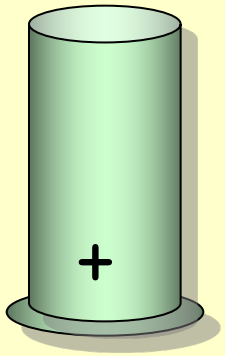




© S. Prasitjutrakul 2013

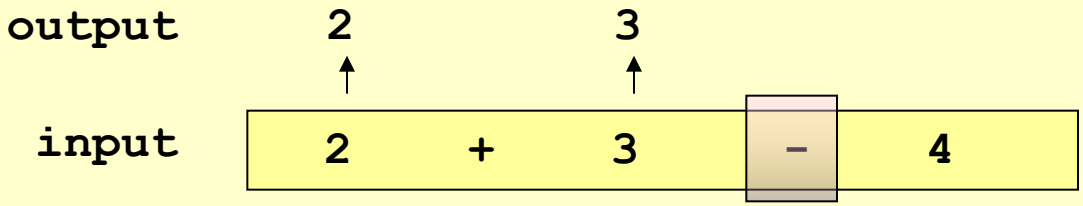
ความสำคัญของ operators

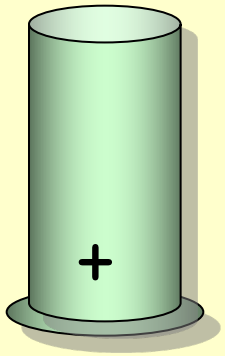




© S. Prasitjutrakul 2013

ความสำคัญของ operators



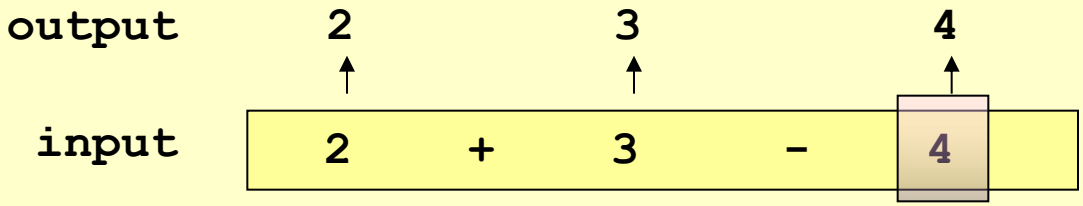


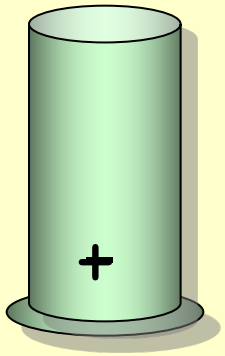
+ มาก่อน
และสำคัญเท่ากับ -

pop ออก ถ้า operator บนกองซ้อน
สำคัญไม่น้อยกว่า operator ใหม่

© S. Prasitjutrakul 2013

ความสำคัญของ operators



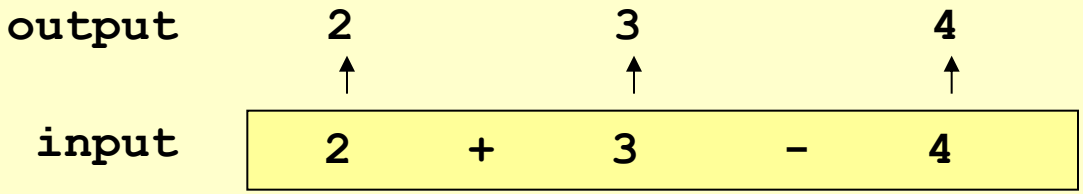


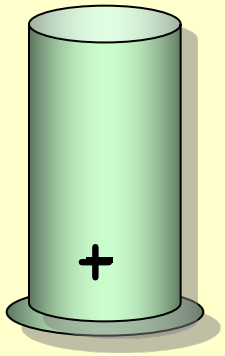
+ มาก่อน
และสำคัญเท่ากับ -

pop ออก ถ้า operator บนกองซ้อน
สำคัญไม่น้อยกว่า operator ใหม่

© S. Prasitjutrakul 2013

ความสำคัญของ operators



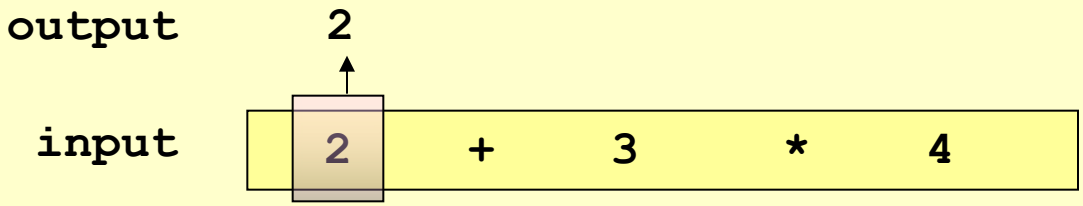


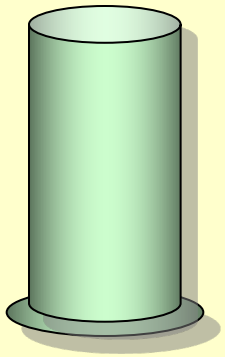
+ มาก่อน
และสำคัญเท่ากับ -

pop ออก ถ้า operator บนกองซ้อน
สำคัญไม่น้อยกว่า operator ใหม่

© S. Prasitjutrakul 2013

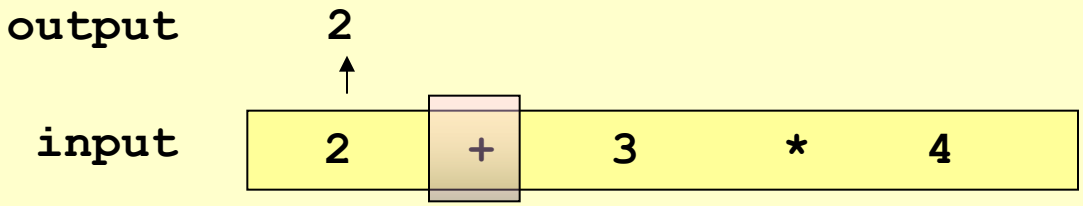
ความสำคัญของ operators

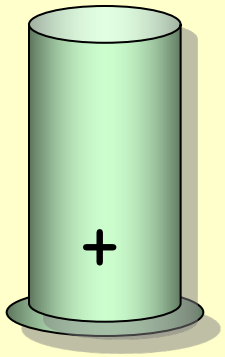




© S. Prasitjutrakul 2013

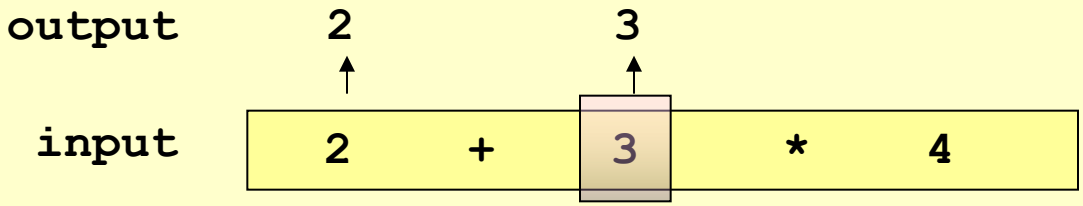
ความสำคัญของ operators

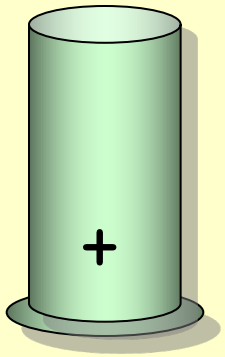




© S. Prasitjutrakul 2013

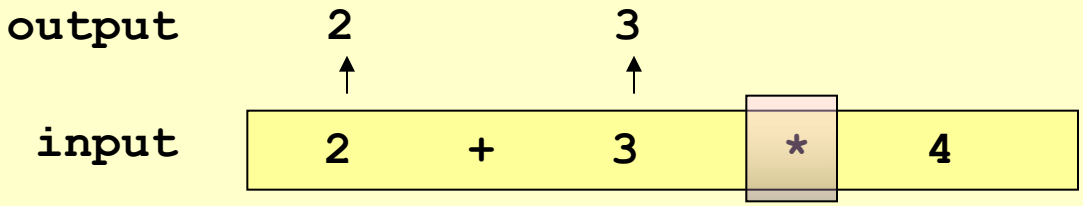
ความสำคัญของ operators

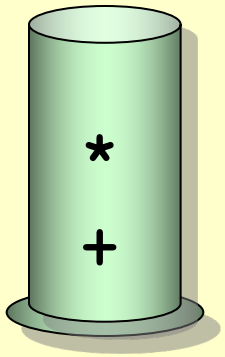




© S. Prasitjutrakul 2013

ความสำคัญของ operators



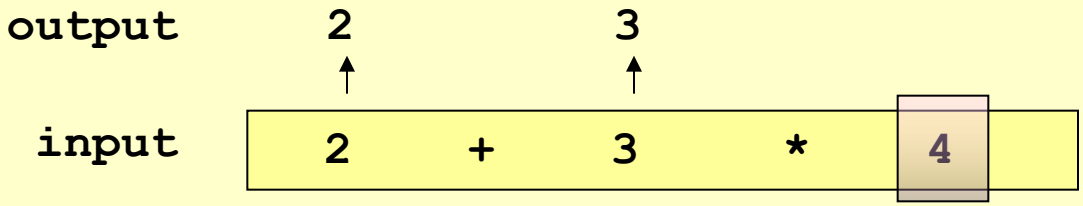


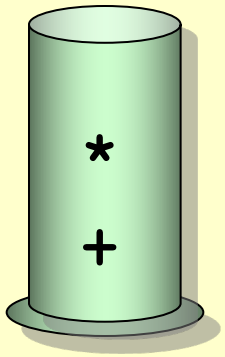
+ มาก่อน แต่ * สำคัญกว่า +

push ทับ ถ้า operator ใหม่สำคัญกว่า operator บนกองซ้อน

© S. Prasitjutrakul 2013

ความสำคัญของ operators



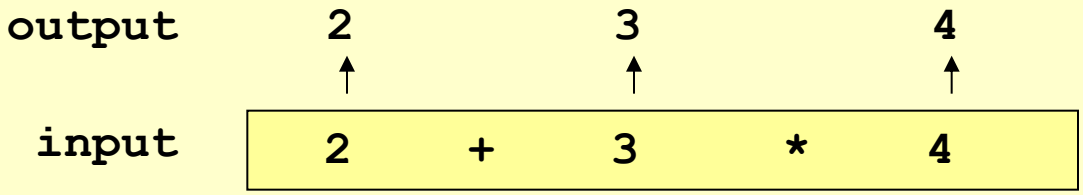


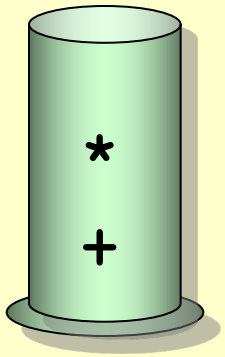
+ มาก่อน แต่ * สำคัญกว่า +

push ทับ ถ้า operator ใหม่สำคัญกว่า operator บนกองซ้อน

© S. Prasitjutrakul 2013

ความสำคัญของ operators





+ มาก่อน แต่ * สำคัญกว่า +

push ทับ ถ้า operator ใหม่สำคัญกว่า operator บนกองซ้อน

© S. Prasitjutrakul 2013

การเปรียบเทียบความสำคัญของ operators

```
string infix2postfix(string &infix) {
  int n = infix.length();
  string postfix = "";
  stack<char> s;
  for (int i=0; i<n; i++) {
    char token = infix[i];
    if (priority[token] == 0) {
      postfix += token;
    } else {
      int p = priority[token];
      while( !s.empty() && priority[s.top()] >= p ) {
        postfix += s.top(); s.pop();
      }
      s.push(token);
    }
  }
  while(!s.empty()) { postfix += s.top(); s.pop();}
  return postfix;
}
```

หาค่าความสำคัญของ operator ตัวใหม่ที่พบ

pop ออก ถ้า operator บนกองซ้อนสำคัญไม่น้อยกว่า operator ใหม่

© S. Prasitjutrakul 2015

การหาความสำคัญของ operators

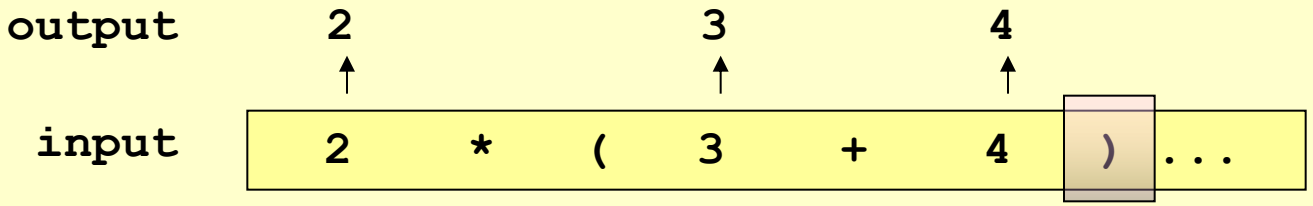
- ^ แทนการยกกำลัง 7
- ^ ทำก่อน + – * /
- *, / ทำก่อน +, –
- * สำคัญเท่ากับ / 5
- + สำคัญเท่ากับ – 3

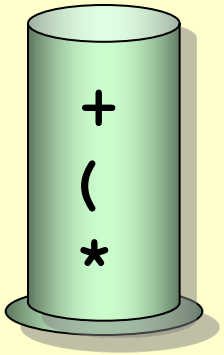
```
map<char,int> priority =
    {{'+',3},{'-',3},{'*',5},{'/',5},{'^',7}};
```

```
int p = priority[token];
while( !s.empty() && priority[s.top()] >= p ) {
  postfix += s.top(); s.pop();
}
```


© S. Prasitjutrakul 2013

ซับซ้อนขึ้นเมื่อมีวงเล็บใน infix

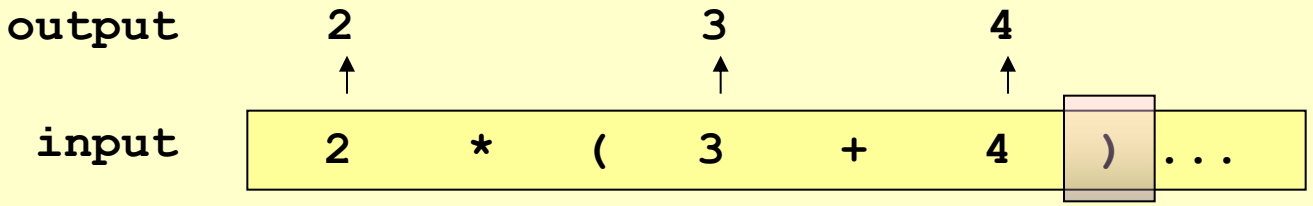


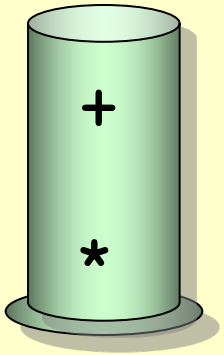


- ภายในวงเล็บเสมือนเป็นนิพจน์ย่อย
- พบวงเล็บเปิด push เสมอ (มีความสำคัญมาก)
- วงเล็บเปิดในกองซ้อน มีความสำคัญน้อยมาก
- พบวงเล็บปิด ลุย pop จนกว่าจะพบวงเล็บเปิด

© S. Prasitjutrakul 2013

ซับซ้อนขึ้นเมื่อมีวงเล็บใน infix

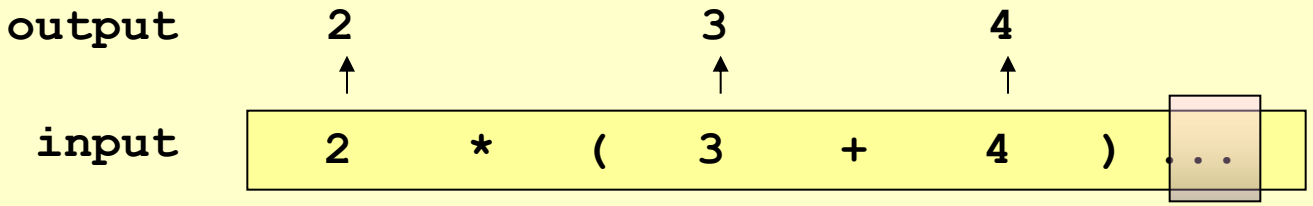


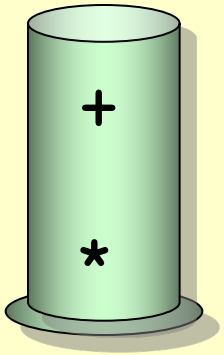


- ภายในวงเล็บเสมือนเป็นนิพจน์ย่อย
- พบวงเล็บเปิด push เสมอ (มีความสำคัญมาก)
- วงเล็บเปิดในกองซ้อน มีความสำคัญน้อยมาก
- พบวงเล็บปิด ลุย pop จนกว่าจะพบวงเล็บเปิด

© S. Prasitjutrakul 2013

ซับซ้อนขึ้นเมื่อมีวงเล็บใน infix





- ภายในวงเล็บเสมือนเป็นนิพจน์ย่อย
- พบวงเล็บเปิด push เสมอ (มีความสำคัญมาก)
- วงเล็บเปิดในกองซ้อน มีความสำคัญน้อยมาก
- พบวงเล็บปิด ลุย pop จนกว่าจะพบวงเล็บเปิด

© S. Prasitjutrakul 2013

ความสำคัญของ operators กรณีมีวงเล็บ

- ขณะพบวงเล็บเปิด
  - มีความสำคัญมาก, จึง push (เสมอ
- แต่พอวงเล็บเปิดอยู่ในกองซ้อน
  - มีความสำคัญน้อยสุด, เพื่อให้ตัวอื่น push ทับ, ยกเว้น)

```
int p = priority[token];
while( !s.empty() && priority[s.top()] >= p ) {
  postfix += s.top(); s.pop();
}
s.push(token);
```

```
int p = outPriority[token];
while( !s.empty() && inPriority[s.top()] >= p ) {
  postfix += s.top(); s.pop();
}
if (token == ')') s.pop(); else s.push(token);
```


© S. Prasitjutrakul 2013

ความสำคัญของ operators กรณีมีวงเล็บ

	+	–	*	/	^	(	)
ขณะที่พบในนิพจน์ (อยู่นอกกองซ้อน)	3	3	5	5	7	9	1
ขณะอยู่ในกองซ้อน	3	3	5	5	7	0	

```
map<char,int> outPriority =
 {{'+',3},{'-',3},{'*',5},{'/',5},{'^',7},{'(',9},{')',1}};

map<char,int> inPriority =
 {{'+',3},{'-',3},{'*',5},{'/',5},{'^',7},{'(',0}};
```


© S. Prasitjutrakul 2013

operator ที่ทำจากซ้ายไปขวา

	+	–	*	/	^	(	)
ขณะที่พบในนิพจน์ (อยู่นอกกองซ้อน)							
ขณะอยู่ในกองซ้อน	3	3	5	5	7	0	

2 + 3 + 4 + 5

2 3 + 4 + 5 +

```
int p = outPriority[token];
while( !s.empty() && inPriority[s.top()] >= p ) {
  postfix += s.top(); s.pop();
}
if (token == ')') s.pop(); else s.push(token);
```


© S. Prasitjutrakul 2013

operator ที่ทำจากซ้ายไปขวา

	+	–	*	/	^	(	)
ขณะที่พบในนิพจน์ (อยู่นอกกองซ้อน)	2	2	4	4	6	9	1
ขณะอยู่ในกองซ้อน	3	3	5	5	7	0	

2 + 3 + 4 + 5

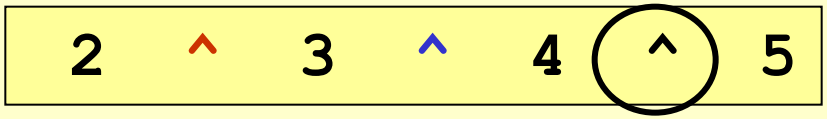
2 3 + 4 + 5 +

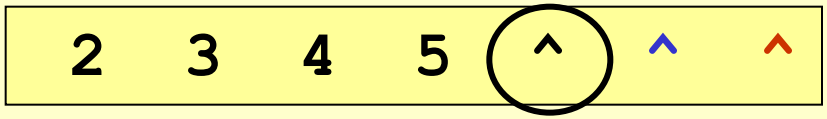
```
int p = outPriority[token];
while( !s.empty() && inPriority[s.top()] >= p ) {
  postfix += s.top(); s.pop();
}
if (token == ')') s.pop(); else s.push(token);
```


© S. Prasitjutrakul 2013

operator ที่ทำจากขวาไปซ้าย

$$2^{3^{4^5}} = 2^{\left(3^{\left(4^5\right)}\right)}$$






```
int p = outPriority[token];
while( !s.empty() && inPriority[s.top()] >= p ) {
  postfix += s.top(); s.pop();
}
if (token == ')') s.pop(); else s.push(token);
```


© S. Prasitjutrakul 2013

operator ที่ทำจากขวาไปซ้าย

$$2^{3^{4^5}} = 2^{\left(3^{\left(4^5\right)}\right)}$$

2	^	3	^	4	^	5

2	3	4	5	^	^	^

```
int p = outPriority[token];
while( !s.empty() && inPriority[s.top()] >= p ) {
  postfix += s.top(); s.pop();
}
if (token == ')') s.pop(); else s.push(token);
```


© S. Prasitjutrakul 2013

operator ที่ทำจากขวาไปซ้าย

$$2^{3^{4^5}} = 2^{\left(3^{\left(4^5\right)}\right)}$$

2	^	3	^	4	^	5

2	3	4	5	^	^	^

```
int p = outPriority[token];
while( !s.empty() && inPriority[s.top()] >= p ) {
  postfix += s.top(); s.pop();
}
if (token == ')') s.pop(); else s.push(token);
```


© S. Prasitjutrakul 2013

operator ที่ทำจากขวาไปซ้าย

$$2^{3^{4^5}} = 2^{\left(3^{\left(4^5\right)}\right)}$$

	+	-	*	/	^	(	)
	2	2	4	4	6	9	1
ขณะอยู่ในกองซ้อน	3	3	5	5	7	0	

2 ^ 3 ^ 4 ^ 5

2 3 4 5 ^ ^ ^

```
int p = outPriority[token];
while( !s.empty() && inPriority[s.top()] >= p ) {
  postfix += s.top(); s.pop();
}
if (token == ')') s.pop(); else s.push(token);
```


© S. Prasitjutrakul 2013

operator ที่ทำจากขวาไปซ้าย

$$2^{3^{4^5}} = 2^{\left(3^{\left(4^5\right)}\right)}$$

	+	-	*	/	^	(	)
	2	2	4	4	8	9	1
ขณะอยู่ในกองซ้อน	3	3	5	5	7	0	

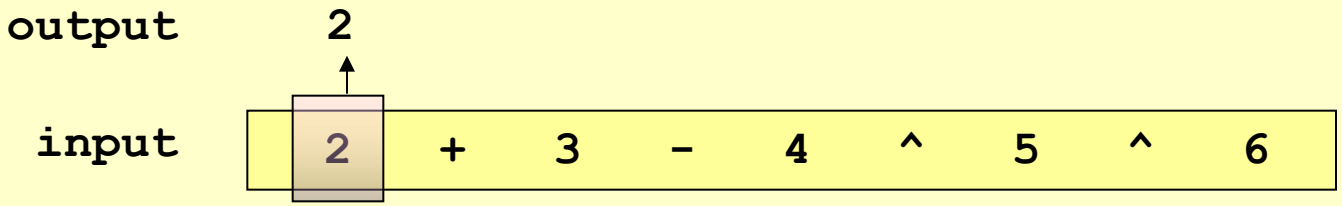
2 ^ 3 ^ 4 ^ 5

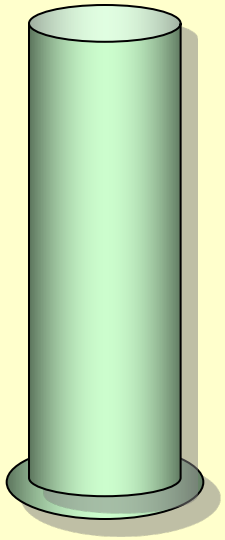
2 3 4 5 ^ ^ ^

```
int p = outPriority[token];
while( !s.empty() && inPriority[s.top()] >= p ) {
  postfix += s.top(); s.pop();
}
if (token == ')') s.pop(); else s.push(token);
```


© S. Prasitjutrakul 2013

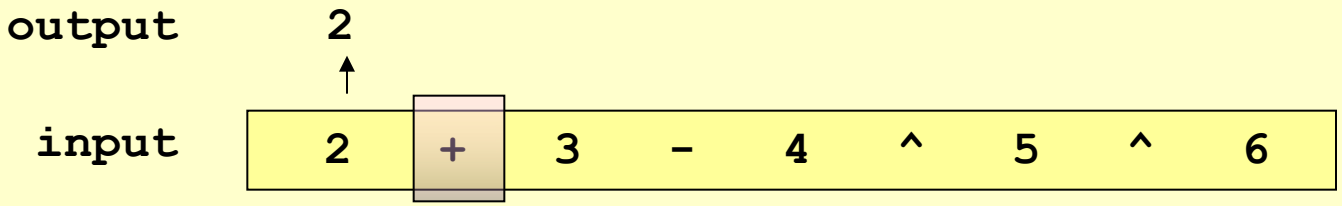
ตัวอย่าง

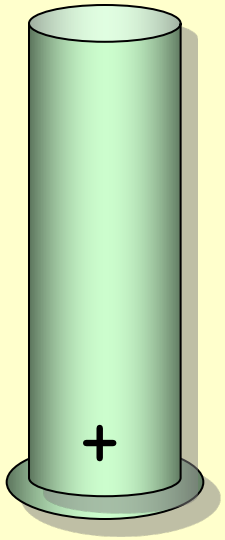




© S. Prasitjutrakul 2013

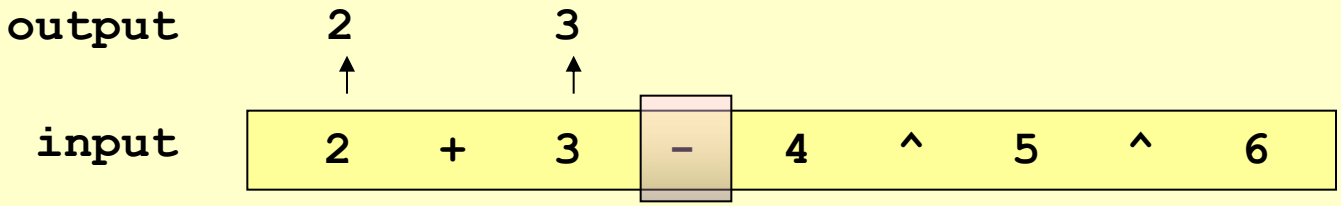
ตัวอย่าง

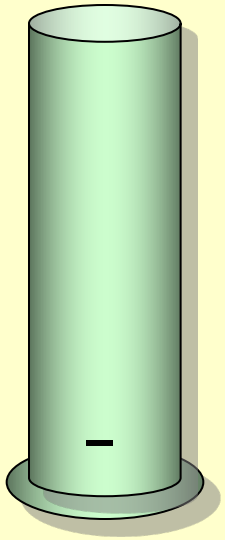




© S. Prasitjutrakul 2013

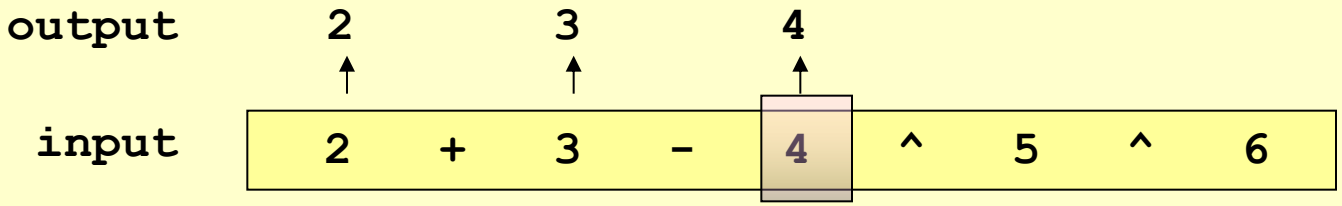
ตัวอย่าง

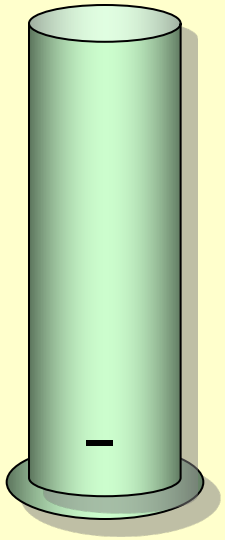




© S. Prasitjutrakul 2013

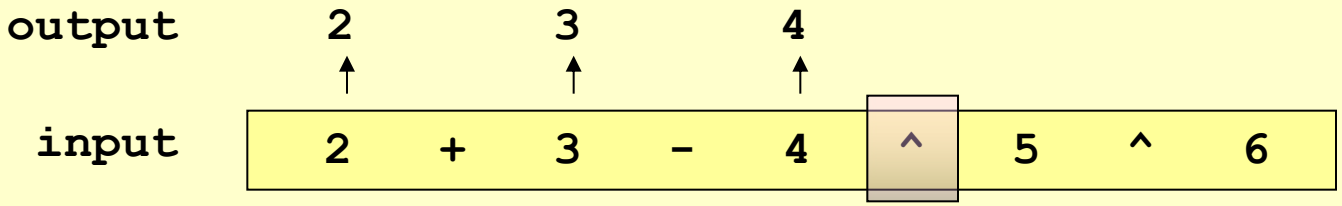
ตัวอย่าง

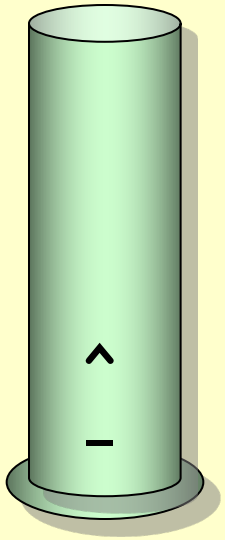




© S. Prasitjutrakul 2013

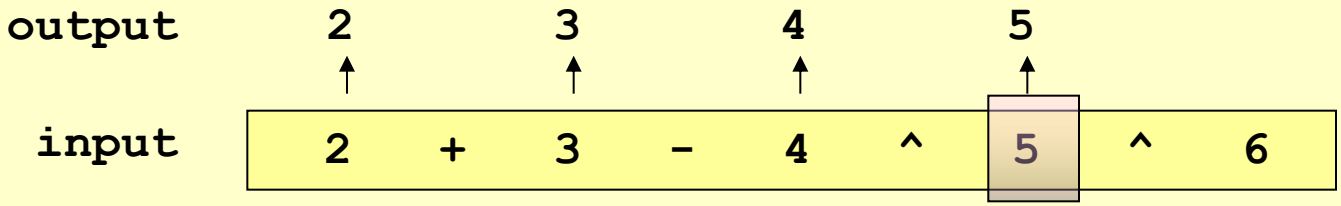
ตัวอย่าง

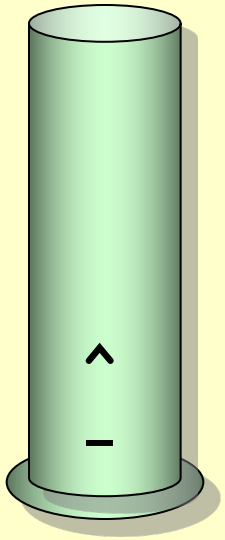




© S. Prasitjutrakul 2013

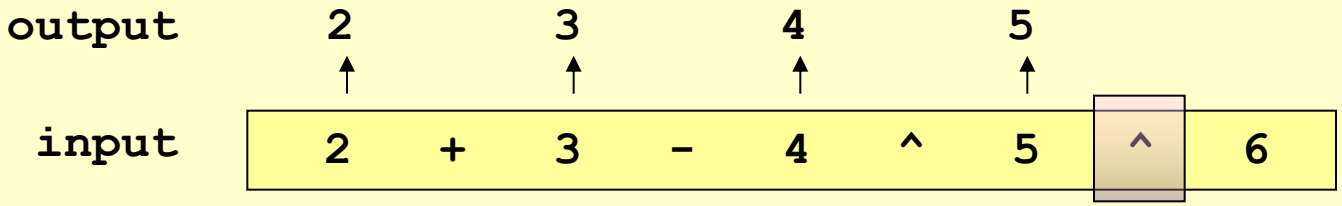
ตัวอย่าง

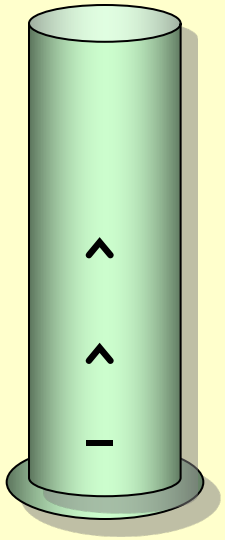




© S. Prasitjutrakul 2013

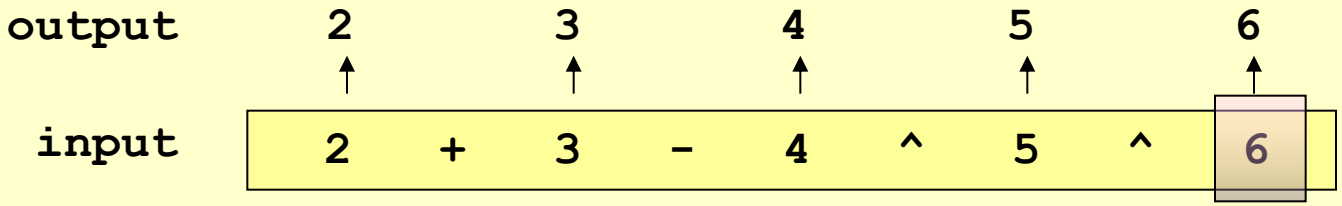
ตัวอย่าง

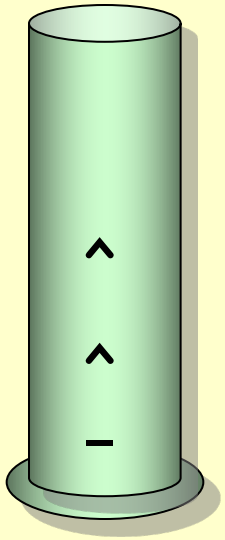




© S. Prasitjutrakul 2013

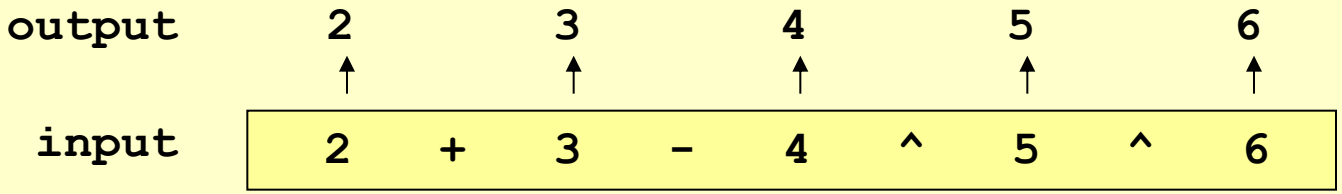
ตัวอย่าง

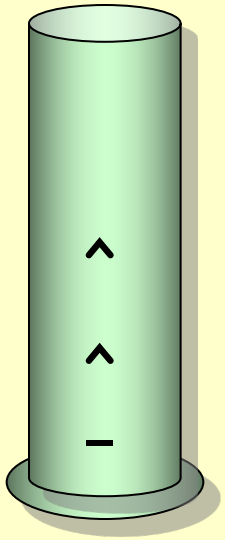




© S. Prasitjutrakul 2013

ตัวอย่าง





© S. Prasitjutrakul 2013

สรุป

- ประยุกต์กองซ้อนในการแก้ปัญหาหลากหลาย
- การดำเนินการหลัก : push / pop / top
- สร้างกองซ้อนได้ง่ายด้วยอาเรย์
- ถ้าจองขนาดให้เพียงพอ การทำงานทุกครั้งเป็น $\Theta(1)$

© S. Prasitjutrakul 2013

แบบฝึกหัด by อ.โต

- เขียนฟังก์ชัน int getKthData(stack<int> s, int k)
  - รีเทิร์นดาต้าตัวที่ k ใน stack s ออกมา (ให้ถือว่าดาต้าบนสุดเป็นดาต้าลำดับที่ 0)
  - ถ้ารีเทิร์นไม่ได้ ให้ throw exception
  - เสร็จแล้ว s ต้องไม่เปลี่ยนหรือมีอะไรหายไป
- เขียนฟังก์ชัน stack<int> reverse(stack<int> s)
  - เรียงลำดับของในสแตกให้กลับกับสแตกต้นฉบับให้หมด
- เขียนฟังก์ชัน double calculate(vector<string> v)
  - ทำการคำนวณหาคำตอบของตัวเวคเตอร์ที่เก็บตัวเลข postfix อยู่แล้ว โดยให้ทำกับ +,-,*,/ เท่านั้น โดยใช้สแตกในการแก้ปัญหา

© S. Prasitjutrakul 2013